IEEE Access
Multidisciplinary ⋮ Rapid Review ⋮ Open Access Journal

Received 10 August 2025, accepted 6 September 2025, date of publication 12 September 2025,
date of current version 22 September 2025.

Digital Object Identifier 10.1109/ACCESS.2025.3609671

RESEARCH ARTICLE

Exploring Deep Learning and Explainable AI for Precise Rice Leaf Disease Detection and Classification

TANJIM MAHMUD[1], (Member, IEEE), MST. SHARMIN AKTER[2], TAHMINA AKTER[2], MOHAMMAD SHAHADAT HOSSAIN[3,4], (Senior Member, IEEE), AND KARL ANDERSSON[4], (Senior Member, IEEE)

[1]Department of Computer Science and Engineering, Rangamati Science and Technology University, Rangamati 4500, Bangladesh
[2]Department of Computer Science and Engineering, Port City International University, Chattogram 4202, Bangladesh
[3]Department of Computer Science and Engineering, University of Chittagong, Chattogram 4331, Bangladesh
[4]Cybersecurity Laboratory, Luleå University of Technology, 931 87 Skellefteå, Sweden

Corresponding authors: Tanjim Mahmud (tanjim_cse@yahoo.com) and Mst. Sharmin Akter (mstsharmin133akter@gmail.com)

ABSTRACT This study presents an innovative approach to detect and classify four types of rice leaf diseases, including bacterial leaf blight, brown spot, tungro, and blast, as well as healthy leaves, using deep learning techniques. By integrating explainable AI methods, such as Grad-CAM, Grad-CAM++, and LIME, our proposed method provides detailed visual explanations of model predictions, enhancing transparency and trustworthiness. Image preprocessing techniques, including Gaussian filtering and Contrast Limited Adaptive Histogram Equalization (CLAHE), were employed to enhance image contrast, while Canny and Prewitt edge detection algorithms were utilized for feature extraction. Experimental results demonstrate that models achieved remarkable accuracy of 99%, significantly outperforming previous methods. However, the study is limited by the insufficient number of rice disease samples. We anticipate that expanding the dataset and increasing the testing scale will further improve the model's generalization capacity and better support agricultural development. The integration of visualization techniques not only improves the interpretability of deep learning models but also facilitates their practical application in precision agriculture.

INDEX TERMS Rice leaf diseases, edge detection, Prewitt, Canny, deep learning, explainable AI.

I. INTRODUCTION

Agriculture acts as a vital part of obtaining food security, reducing poverty, and strengthening development [1]. One of the most important crops for humans is rice, which is essential to sustaining the food supply [2]. Rice is exposed to both biotic and abiotic stressors throughout its development [3]. One of the main threats to rice is disease, which may lead to significant losses in output and quality. There are various diseases threatening rice growth, such as bacterial blight, blast,tungro, and brown spot. The traditional method for the detection of rice diseases is often carried out under the guidance of laboratory testing or visual examination [4]. Only a skilled individual can do the time-consuming visual assessment. Leaves exhibit certain disease related signs when there is a disease present on the leaves [5]. In order to identify the exact diseases, these distinct symptoms serve as a characteristic. Timely and accurate detection of these diseases is imperative for implementing effective management strategies to mitigate them. Advancements in deep learning, a subset of artificial intelligence, offer promising solutions to these challenges. Deep learning techniques have demonstrated exceptional performance in image recognition tasks [6],making them suitable for identifying and classifying diseases from leaf images [7], [8]. Accurate classification enables timely and appropriate intervention strategies, mitigating disease impact [9]. Targeted use of pesticides may lower total chemical

The associate editor coordinating the review of this manuscript and approving it for publication was Jeon Gwanggil.

© 2025 The Authors. This work is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 License.
For more information, see https://creativecommons.org/licenses/by-nc-nd/4.0/

consumption and encourage environmentally friendly agricultural methods when diseases are accurately diagnosed. In order to increase rice yield and protect crops from diseases and pests, it is vital to create a novel approach for the swift and precise identification of rice diseases at their early stages [10]. This proactive detection is critical for maintaining both the quality and productivity of rice. The aim of this research is to design a reliable and accurate diagnostic system by exploring the use of deep neural network models to detect diseases that impact rice leaves. In order to identify disease patterns from digital images of rice leaves, the study focuses on creating and training deep learning-based models. These algorithms are able to detect particular diseases with high accuracy and discriminate between healthy and sick leaves by using massive datasets and sophisticated image processing methods. This study also highlights how crucial visualization tools are to improving the usability and interoperability of DL models [11]. The models' decision-making process may be made more understandable with the use of visualization approaches, which can also provide important insights into the characteristics that influence disease diagnosis [6]. Through enhancing the early identification and diagnosis of diseases, this research advances sustainable agricultural methods,guaranteeing increased rice crop production and quality, and eventually promoting global food security.

A. OBJECTIVES OF THIS STUDY

1) To develop an effective model to automatically detect rice leaf diseases.
2) To create a reliable and accurate model for visualization.
3) To develop visualization techniques to highlight distinctive visual patterns and symptoms associated with various rice leaf diseases.

B. CONTRIBUTIONS OF THIS STUDY

1) The study concentrates on detecting four specific rice leaf diseases: bacterial leaf blight, brown spot, tungro, and blast, as well as identifying healthy leaves.
2) Preprocessing methods Uses Gaussian filter and CLAHE (Contrast Limited Adaptive Histogram Equalization) to improve image contrast.
3) Edge Detection Techniques utilizes Canny edge detection and Prewitt edge detection to enhance model prediction.
4) Explainable AI Integration employs Grad-CAM, Grad-CAM++, and LIME to provide detailed and accurate visual explanations of model predictions.

II. REVIEW OF PRIOR RESEARCH

The integration of AI and the Internet of Things (IoT) [12] has significantly transformed the agricultural sector, enhancing productivity, crop management, and disease detection.

Sethy et al. [13] used deep neural networks and SVM for identifying rice leaf diseases, extracting deep features from CNN models. Their approach, which included a dataset of 5932 images, showed that ResNet50 with SVM performed best.

Feng et al. [14] employed hyperspectral imaging (HSI) and DL methods to detect rice leaf diseases across different varieties. Fine-tuning was found to be the most effective approach.

Mekha and Teeyasuksaet [15] compared various classification algorithms for rice leaf disease detection, concluding that the RF algorithm outperformed others in terms of accuracy.

Tejaswini et al. [2] introduced a rice leaf disease classification method using CNN, aimed at early detection and diagnosis of diseases to implement timely preventative measures. Their approach, which classifies four rice diseases using 1600 images, integrates a custom 5-layer CNN alongside popular DL models such as VGG19, VGG16, Xception, and ResNet. Among these, the 5-layer CNN demonstrated the best performance (see Table 1).

Zeng et al. [4] proposed a precise rice disease categorization method based on ICAI-V4, enhanced by the Candy algorithm to reduce image noise while highlighting edge features. This method, which uses a coordinated focus strategy, outperformed other models like AlexNet and ResNet50 in accuracy when validated with 10-fold cross-validation on 10,241 images.

Simhadri and Kondaveeti [3] applied transfer learning to automatically identify rice leaf diseases using 15 pre-trained CNN models. This method fine-tunes a model from a large dataset to a smaller, task-specific one, improving disease detection accuracy.

Aggarwal et al. [17] proposed a deep learning-based method for forecasting rice leaf diseases. By utilizing 32 pre-trained models and ensemble learning classifiers, their approach achieved an accuracy rate of 0.94 with the EfficientNetV2B3 model, demonstrating its effectiveness in diagnosing rice diseases.

Rajpoot et al. [19] explored the use of Faster R-CNN deep architecture and VGG-16 for detecting three key rice diseases. Their method, supported by a random forest classifier, achieved a 0.97 prediction accuracy, proving to be an efficient solution for disease detection.

Sudhesh et al. [16] introduced the Dynamic Mode Decomposition (DMD) approach, tested with 10 deep CNN models using transfer learning. DenseNet121 outperformed other models in disease localization, highlighting the effectiveness of this method.

Ahad et al. [24] compared CNN-based architectures for classifying nine common rice diseases. Their study used ensemble models and transfer learning on DenseNet121, MobileNetV2, and others, finding that the ensemble DEX model performed best.

Junaidi et al. [25] assess training data from 5932 images of rice leaf disease using 17 transfer learning models. The findings demonstrate that the ensemble model's accuracy

TABLE 1. Summary of studied research works.

Reference	Crop	Dataset	Classifier	Accuracy	Key Insights
[2]	Rice	1600 images	VGG19,VGG16, Xception, Resnet, 5-layer convolutional network	0.78	This paper's main goal is to investigate, determine the disease name, and diagnose rice leaf diseases beforehand so that appropriate safety precautions can be implemented.
[4]	Rice	10,241 images	AlexNet, ResNet50, Inceptionv4, DenseNet121, ResNetXt, and MobileNetv3, ICAI-V4 model	0.95	This research provides an enhanced Canny algorithm and ICAI-V4 model-based approach for diagnosing and categorizing rice diseases. In order to minimize picture noise and emphasize edge features, the suggested technique first applies an algorithm based on the Canny algorithm.
[3]	Rice	100,80 images	Xception, InceptionResNetV2, ShuffleNet, SqueezeNet, DarkNet-53,DenseNet 201,	0.97	To categorize up to 10 rice leaf diseases and researchers created a deep transfer learning(TL) model. They classified a healthy class as well. Of the 15 pretrained deep CNN models, the best successful model was found by the use of TL to train the model on pre-trained models.
[16]	Rice	3416 images	10 transfer-learned Deep CNN,XceptionNet deep feature with SVM	0.94	Dynamic Mode Decomposition (DMD) is presented in this research. Ten transfer-learned Deep CNN models were tested in four different tests. Three ML models were trained using the deep features, and DenseNet121 performed better than the other models.
[17]	Rice	551 images	EfficientNetV2B3 with ET and HGB	0.94	The proposed EfficientNetV2B3 with ET and HGB classifier achieved 0.90-0.91 identification accuracy and other performance parameters on a normal dataset.
[18]	Rice	3500 images	CNN	0.70	The study focused on the difficulty farmers encounter in recognizing diseases in rice fields, an essential aspect of efficient crop management. In order to preserve crop health and production, this technique was designed to help farmers effectively spot diseases in paddy crops.
[19]	Rice	120 images	R-CNN deep architecture and VGG-16 transfer learning	0.97	Investigated three main diseases affecting rice. It proposes a method using Faster R-CNN architecture and VGG-16 to extract features and categorize attributes. The RF classifier splits the radish field into three sections.

TABLE 2. Summary of studied research works is related to rice leaf diseases.

Reference	Crop	Dataset	Classifier	Accuracy	Key insights
[20]	Tomato	11123 images	RF, SVM, NB, CNN	0.95	CNN using a pre-trained Inception v3 model was superior than conventional techniques in terms of detection and classification.
[21]	Tea	5280 images	DCNN	0.94	A novel DCNN proposed for diagnosing grey blight disease on tea leaves.
[22]	Potato	600 images	CNN	0.97	A CNN-based method is proposed for predicting potato leaf diseases using a pre-trained model. The method uses a dataset of potato leaf images, including healthy and diseased leaves.
[23]	Apple	700 inages	Resnet50, ShuffleNet, MobileNet, MobileNetV2	0.96	The researchers improved an algorithm for identifying apple leaf disease using MobileNetV2, incorporating ECA-Net attention, feature reuse, and dense connection structure, and training on a larger dataset.

increased significantly from 0.35 to 0.97, underscoring the possibility of combining several architectures to maximize classification accuracy.

Chaudhari and Malathi [26] developed a CNN-SVM hybrid model that has been created to identify and address damaged leaves. Images taken in agricultural areas, processed for quality, and separated using GrabCut are used in the model. The Inception-ResNet V2-SVM technique is used to extract and classify features. Outperforming earlier approaches, the model obtained accuracy, precision, recall, and error values of 0.97, 0.93, and 0.03, respectively Bhimavarapu [27] developed a CNN that can efficiently diagnose diseases by extracting features from images. The study proposes an improved activation and optimizer function (IAOF) in the CNN model to minimize loss and improve prediction performance.

Mannepalli et al. [28] present research on a rice leaf disease diagnosis method using the VGG16 convolutional neural network, achieving 97.77% classification accuracy on a publicly available dataset, demonstrating its potential for precision agriculture.

Stephen et al. [29] developed a 3D2D DCNN for rice disease classification, combining a 2DCNN for features and an improved backtracking search algorithm, achieving 98.7% accuracy.

Kaur et al. [30] proposes a DL-based approach for rice leaf disease detection using pre-trained models such as VGG16, SqueezeNet, and InceptionV3. Among them, SqueezeNet achieves the highest accuracy of 93.3%. The model aims to facilitate precise and timely interventions to mitigate ecological and economic impacts.

Kulkarni and Shastri [31] explores the application of image processing and ML for rice leaf disease detection. A newly proposed CNN model achieves 95% accuracy in classifying common diseases under various lighting and background conditions, highlighting the potential of machine learning in supporting agricultural disease management.

Dai et al. [32] introduced AISOA-SSformer, a Transformer-based semantic segmentation algorithm. It incorporates a sparse global-update perceptron for real-time parameter updating, a salient feature attention mechanism, and an annealing-integrated sparrow optimization algorithm. Experimental results show that achieves an 83.1% MIoU, 80.3% Dice coefficient, and 76.5% recall, demonstrating superior accuracy.

Mukherjee et al. [33] provides a comprehensive review of techniques for rice leaf disease identification and classification. The study covers visual characteristics, imaging modalities, image acquisition techniques, image processing methods, feature extraction, machine learning algorithms, and mobile applications for real-time disease detection.

Sagar and Singh [20] evaluated the use of CNN, specifically Inception v3, for classifying tomato leaf diseases. Their model achieved over 0.95 accuracy, outperforming traditional methods (see Table 2).

Pasalkar et al. [22] demonstrated a CNN-based approach using pre-trained models to classify potato leaf diseases, achieving 0.97 accuracy, making it an effective tool for early disease detection. Liu et al. [23] developed an enhanced MobileNetV2 model for apple leaf disease detection, achieving 96.23% accuracy, outperforming other models like ResNet50 and ShuffleNet.

III. METHODS

An overview of the methods approach utilized to carry out the study and analyze the data is given in this section.

A. DATASETS

This study primarily utilized datasets sourced from open-source platforms on Kaggle: (i)Rice Leaf Disease Image Samples,[1] (ii) rice_leaf_images.[2] We created a new Rice Leaf Disease dataset by combining two existing datasets: Dataset (i), which contains images of four types of rice leaf diseases(See Table 3 and Figure 1) but lacks healthy leaf images, and Dataset (ii), which provides images of healthy leaves. By adding the healthy leaf images from Dataset (ii) to the disease images in the dataset (i), we formed a comprehensive dataset with a total of 7,420 images, distributed as follows: 1,584 images of Bacterial Blight, 1,440 images of Blast, 1,600 images of Brown Spot, 1,308 images of Tungro, and 1,488 images of Healthy leaves.

B. DATA PREPROCESSING

Data preprocessing is a crucial step in the data analysis pipeline, involving the transformation, cleaning, and organization of raw data to ensure its suitability for analysis or modeling [6]. This process includes various tasks to enhance the quality, consistency, and usability of the data. In this dataset, all images are resized to 224 × 224x3. Several standard techniques are applied to preprocess the rice leaf disease dataset.

1) GAUSSIAN FILTER

The Gaussian Filter, also referred to as Gaussian smoothing or Gaussian blur, is a popular image processing tool used to reduce noise and blur images (see Figure 2). This filter helps soften certain areas of an image or diminish high-frequency components. It operates using an odd-sized symmetric kernel, which is applied to each pixel within the region of interest. The filter gives more weight to the pixels at the center of the kernel, making it less responsive to abrupt color changes at the edges, resulting in smoother transitions in the image [35]. The algorithmic approach of the Gaussian Filter is as follows:

1) Create *output_folder* if it doesn't exist.
2) For each *class_folder* in *input_folder*:
   a) Create *class_output_folder* in *output_folder* if it doesn't exist.
   b) For each *filename* in *class_folder*:
      i) Set *input_path* and *output_path*.
      ii) If *input_path* is a file:
         A) Open the image.
         B) Convert image to *'RGB'* if it is in *'RGBA'*.
         C) Convert image to NumPy array (*img_array*).
         D) Apply Gaussian filter to each channel of *img_array*. σ = 1.5 and filter size is 11 × 11.
         E) Convert filtered array back to an image.
         F) Save the filtered image to *output_path*.
3) End.

[1] https://data.mendeley.com/datasets/fwcj7stb8r/1
[2] https://www.kaggle.com/datasets/nizorogbezuode/rice-leaf-images

TABLE 3. Summary of rice leaf diseases.

Disease	Cause	Symptoms and Effects
Bacterial Blight	Caused by the bacterium *Xanthomonas*	Water-stained lesions appear at the leaf border near the tip and spread towards the leaf base. Affected areas become pale brown and dry out, expanding and lengthening [34].
Blast	Rice blast disease, identified by green-gray spots with dark green edges on leaves	As the disease progresses, oval or spindle-shaped lesions with reddish-brown margins develop, sometimes taking a rhombus shape. Lesions enlarge and may merge, leading to leaf drop [4].
Brown Spot	Caused by *Bipolaris oryzae*	Characterized by large spots on leaves that may kill the entire leaf. Infection in seeds causes discolored or unfilled grains. Seedlings may show small, round, yellow-brown lesions, potentially girdling the coleoptile and stunting primary and secondary leaves.
Tungro	Caused by the Green Leaf Hopper and *Nephotettix virescens*	Results in discolored leaves (yellow or orange-yellow), slowed growth, fewer tillers, and empty grains. Affected plants exhibit necrosis, rust-colored patches, and a mottled or striped appearance.

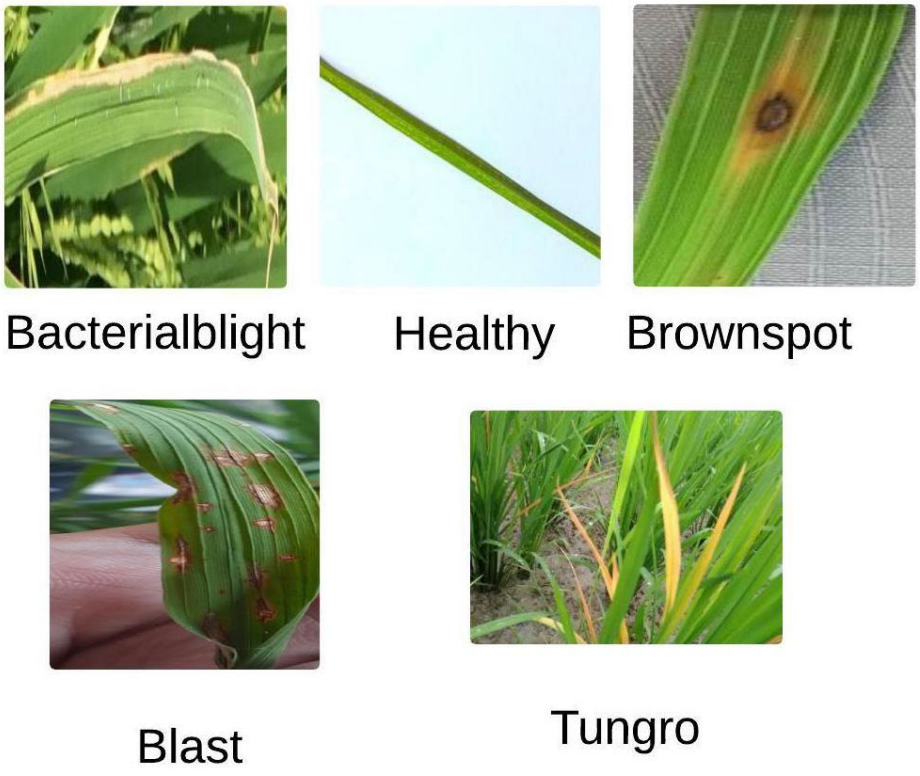


FIGURE 1. Some samples of rice leaf diseases.

2) CONTRAST LIMITED ADAPTIVE HISTOGRAM EQUALIZATION (CLAHE)

The main purpose of CLAHE is to improve an image's contrast through preprocessing. It modifies the image's intensity distribution to enhance visual appeal and enhance the visibility of features. The image's brightness values are redistributed using a number of histograms that are computed, each of which represents a different area of the image(See Figure 3). As such, it's appropriate for boosting edge definitions and local contrast in every area of an image [36].

The algorithmic steps of CLAHE is as follows:

1) Create *output_folder* if it doesn't exist.
2) For each *class_folder* in *input_folder*:
    a) Create *class_output_folder* in *output_folder* if it doesn't exist.

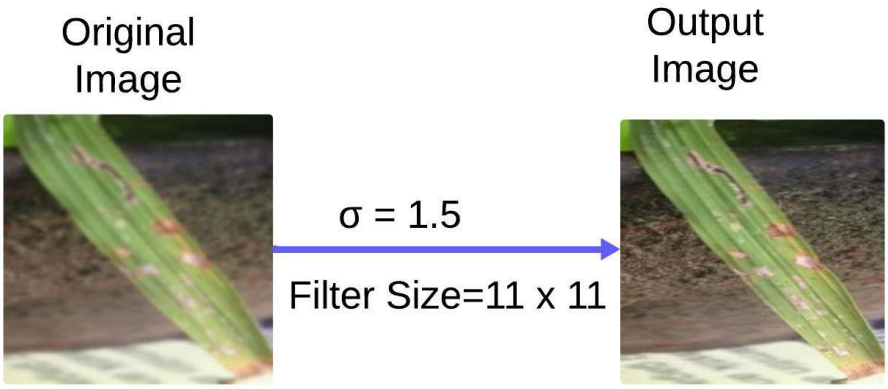


FIGURE 2. After gaussian filter.

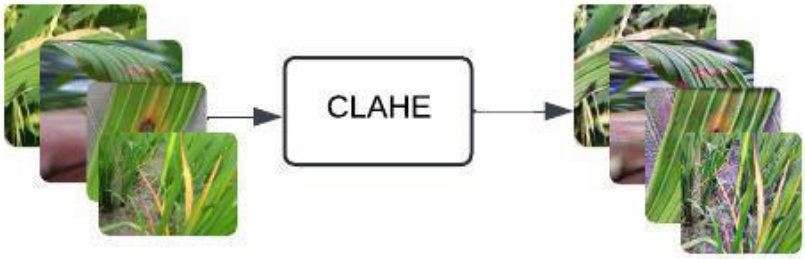


FIGURE 3. After apply CLAHE.

b) For each *filename* in *class_folder*:
- Set *input_path* and *output_path*.
- If *input_path* is a file:
  - Open, convert, and process image:
    i) Open and convert to 'RGB'.
    ii) Convert to NumPy array (*img_array*).
    iii) Apply CLAHE to each channel.
    iv) Convert back to image and save.

3) End.

C. FEATURE EXTRACTION

This process transforms the original features into a new set, retaining essential information and emphasizing key patterns. Two methods are utilized for feature extraction: (i) Canny Edge Detection and (ii) Prewitt Edge Detection.

We experimented with different combinations of these edge detection methods along with preprocessing techniques. Gaussian filtering smooths the image before edge detection, helping remove noise while preserving significant disease-related features. CLAHE (Contrast Limited Adaptive Histogram Equalization), on the other hand, enhances local contrast, making diseased regions more prominent before extracting edges. By combining these techniques with both Canny and Prewitt filters, we analyzed their effects on the performance of various deep learning models, including CNN, ResNet, MobileNet, VGG, and Inception.

1) CANNY EDGE DETECTION

Canny edge detection uses linear filtering with a Gaussian kernel to smooth noise, determining edge strength and direction for each pixel in a smoothed picture. Eligible pixels pass through non-maximal suppression, with edge intensity set at zero(See Figure 4). Hysteresis thresholds the thinned edge magnitude image, employing two edge strength thresholds. Users must enter three settings: sigma, low, and high, representing the Gaussian filter's standard deviation and gradient magnitude distribution [37].

The solving steps of Canny Edge Detection is as follows:

1) Check if the *output_path* directory exists; if not, create it.
2) Define a function `apply_canny_edge_detection_rgb` to apply Canny edge detection on each channel of an RGB image.
3) Iterate over each *class_folder* in the *dataset_path*: i.

   a) Construct the path to the current *class_folder*.
   b) For each *image_name* in the *class_folder*: A.

      i) Construct the full *image_path*.
      ii) Read the image using OpenCV (`cv2.imread`).
      iii) Apply Canny edge detection to the RGB image using `apply_canny_edge_detection_rgb` function.

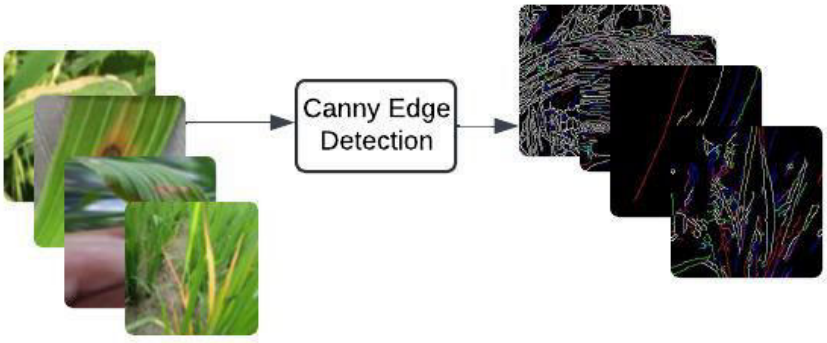


FIGURE 4. After canny edge detection.

iv) Define *output_image_path* to save the processed image in the *output_path* directory, maintaining the folder structure of the dataset.
v) Create necessary directories in *output_image_path* if they don't exist.
vi) Write the processed edges image to *output_image_path* using `cv2.imwrite`.

4) End.

2) PREWITT EDGE DETECTION

Prewitt edge detection is to detect edges in digital images(see Figure 5). This kind of gradient-based edge detection technique determines the gradient of the pixel-by-pixel picture intensity function. Two 3×3 convolution kernels make up the Prewitt operator, one is used to detect vertical edges and the other to detect horizontal ones. To detect the edges, these kernels are applied to the picture independently and the results merged [38].

The way of Prewitt Edge Detection is as follows:

1) Check if `output_path` directory exists; if not, create it.
2) Define `apply_prewitt_edge_detection_rgb` function to apply Prewitt edge detection on each channel of an RGB image.
3) Iterate over each `class_folder` in `dataset_path`:
   a) Construct the path to the current `class_folder`.
   b) For each `image_name` in the `class_folder`:
      i) Construct the full `image_path`.
      ii) Read the image using OpenCV (`cv2.imread`).
      iii) Apply Prewitt edge detection using `apply_prewitt_edge_detection_rgb`.
      iv) Define `output_image_path` to save the processed image in `output_path`, maintaining the dataset structure.
      v) Create necessary directories in `output_image_path` if they don't exist.
      vi) Write the processed edges image to `output_image_path` using `cv2.imwrite`.

D. IDENTIFICATION OF RICE DISEASES BASED ON MODELS

A deep learning model is an advanced machine learning approach that leverages multi-layered neural networks to automatically extract and learn complex patterns from data [39]. The following steps summarize the proposed deep Learning Models(See Figure 6):

Step 1: Collection of Rice Leaf Diseases Images

Step 2: Pre-processing the Images

- Image resizing: Resize images to $224 \times 224 \times 3$ dimensions to ensure uniform input for the model.
- Gaussian filter: Apply a Gaussian filter to reduce high-frequency noise in the images, smoothing out irrelevant details.
- CLAHE (Contrast Limited Adaptive Histogram Equalization): Enhance image contrast using CLAHE to improve feature visibility.

Step 3: The data is processed to extract features, particularly patterns or structures seen in images that correspond to rice leaf diseases. This uses edge detection methods to find edges in the images, such as Canny and Prewitt Edge Detection algorithms.

Step 4: Classification is performed using deep learning models and measures performance.

E. EXPLAINABLE AI METHODS

Artificial intelligence (AI) systems and models that are intended for human comprehension are referred to as explainable AI (XAI) [40], [41]. Explainable AI works by applying various methods to shed light on the decision-making process of complex AI models. These methods can pinpoint the most influential data features, analyze how individual predictions are made, and even highlight specific parts of an image that an AI relied on for its classification. By using these techniques, XAI helps us understand the reasoning behind an AI's output, fostering trust and enabling improvements.

In this study, we utilized Explainable AI (XAI) methods to enhance the interpretability of deep learning models

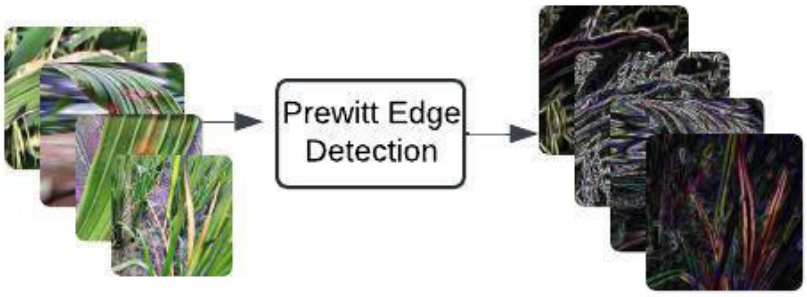


FIGURE 5. After prewitt edge detection.

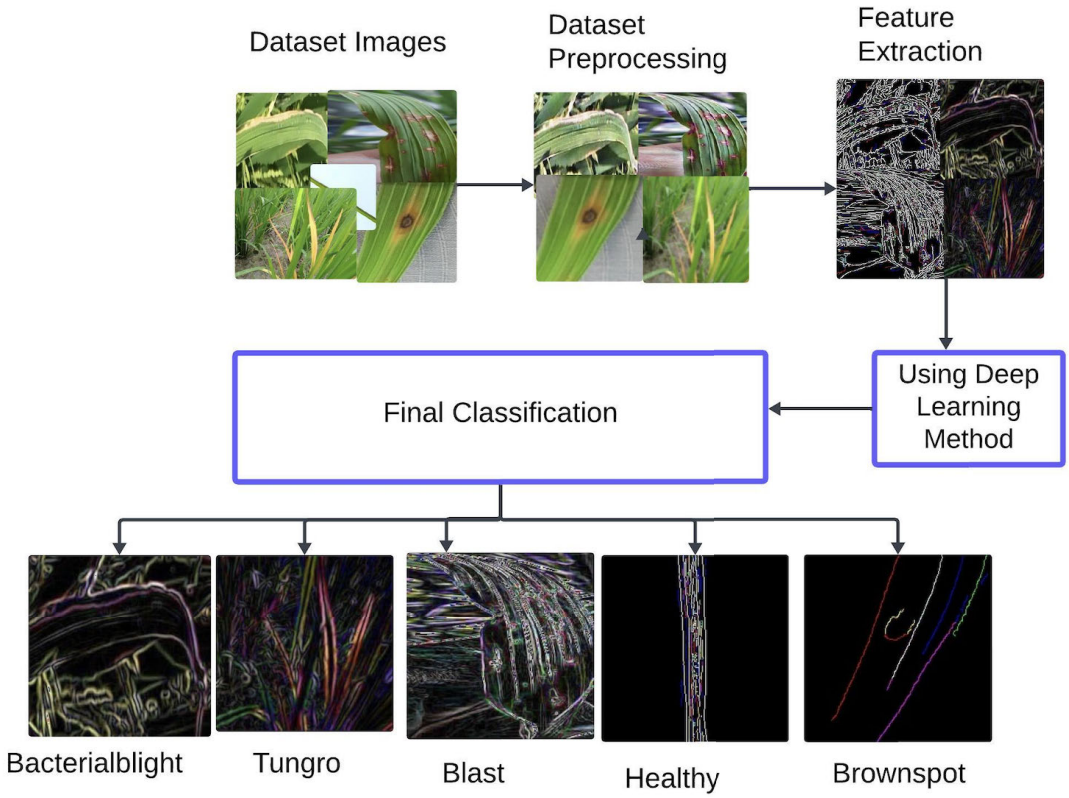


FIGURE 6. System architecture.

(See Figure 7). Specifically, we employed Grad-CAM and Grad-CAM++ for visualizing the most influential regions in an image that contribute to the model's predictions. These methods were implemented using the TensorFlow and Keras libraries. Grad-CAM generates a heatmap by computing gradients of the target class score with respect to the feature maps of the last convolutional layer, highlighting the most relevant areas of the image. Grad-CAM++ further improves this approach by considering higher-order derivatives, allowing for finer localization of important regions. Additionally, we incorporated Local Interpretable Model-agnostic Explanations(LIME) for image classification explainability. LIME was implemented using the lime Python package, which perturbs an input image and observes the effect on the model's predictions, thereby identifying the most important image regions that influence the decision. The LIME-generated masks highlight these critical areas, providing an intuitive understanding of how the model makes predictions. To refine the LIME explanations, we utilized OpenCV for post-processing, including dilation and contour detection, to better visualize the identified features.

1) GRAD-CAM

Grad-CAM is a technique that helps to show the parts of an image that a CNN uses to decide which class to classify [42], [43]. Understanding and interpreting the model's predictions might be aided by the knowledge it offers regarding the

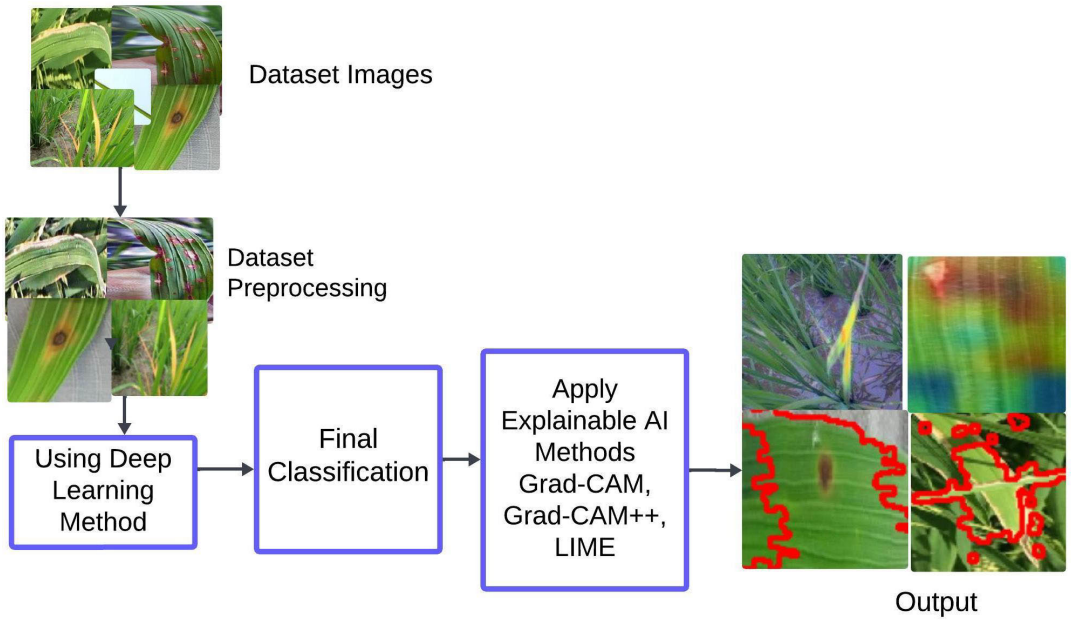


FIGURE 7. Explainable AI integration.

network's primary emphasis. The Grad-CAM procedure is explained in detail below:

- Forward Pass: Let $f(X)$ be a deep convolutional neural network where X is the input image. The output of the last convolutional layer before classification is represented as:

$$A^k \in \mathbb{R}^{u \times v} \tag{1}$$

  where A^k denotes the activation map of the k-th feature map, and u, v are spatial dimensions.
- Pass (Gradient Computation): To understand the importance of each feature map, we compute the gradient of the class score y^c (corresponding to class c) with respect to the feature map activations:

$$\frac{\partial y^c}{\partial A^k} \tag{2}$$

  These gradients indicate how much each spatial location in the feature maps influences the class prediction.
- Weight Calculation: The importance weight α_k^c for each feature map A^k is obtained by computing the global average of the gradients over all spatial locations:

$$\alpha_k^c = \frac{1}{Z} \sum_i \sum_j \frac{\partial y^c}{\partial A_{ij}^k} \tag{3}$$

  where $Z = u \times v$ is the total number of pixels in the feature map.
- Heatmap Generation: Using these weights, the Grad-CAM localization map L^c is obtained as:

$$L^c = \text{ReLU}\left(\sum_k \alpha_k^c A^k \right) \tag{4}$$

  The ReLU function ensures that only positive contributions are considered, removing negative values that may suppress relevant activations.
- Superimposition: The heatmap L^c is then upsampled to match the original image size and overlaid for visual interpretation:

$$\hat{L}^c = \text{Upsample}(L^c) \tag{5}$$

  where $\hat{L}^c$ represents the final heatmap superimposed on the original image, helping interpret the model's decision-making process.

By following these analytical steps, Grad-CAM provides a transparent and explainable mechanism for deep learning-based classification tasks. The heatmaps generated by Grad-CAM allow researchers and practitioners to visualize important regions of the image that contribute to model predictions, thereby enhancing model trustworthiness and interpretability.

-CAM++ A more precise and comprehensive visual explanation of CNN predictions is offered by Grad-CAM++, an improved version of Grad-CAM [44]. Grad-CAM++ enhances this by resolving some constraints and improving the heatmap, particularly in cases with numerous objects or fine-grained features, while Grad-CAM generates a coarse localization map showing the essential areas for a particular class. The Grad-CAM++ procedure is explained in detail below:

- Forward Pass: The input image X is passed through the network, and activations A^k from the last convolutional layer are extracted:

$$A^k \in \mathbb{R}^{u \times v} \tag{6}$$

TABLE 4. Model training configurations.

Model	Batch Size	Epoch	Execution Time	Configuration
Gaussian+Canny+CNN	32	20	637s	GPU:V100 RAM:High-RAM
Gaussian+Canny+ResNet50	32	20	1192s	GPU:TPU RAM:High-RAM
Gaussian+Canny+MobileNet	32	10	552s	GPU:V100 RAM:High-RAM
Gaussian+Canny+MobileNetv2	32	25	800s	GPU:V100 RAM:High-RAM
Gaussian+Canny+VGG19	32	20	827s	GPU:TPU RAM:High-RAM
Gaussian+Canny+InceptionV3	32	25	552s	GPU:V100 RAM:High-RAM
Gaussian+Canny+VGG16	32	20	717s	GPU:TPU RAM:High-RAM
Gaussian+Canny+ResNet101	32	25	1263s	GPU:V100 RAM:High-RAM
CLAHE+Canny+CNN	32	30	617s	GPU:V100 RAM:High-RAM
CLAHE+Canny+ResNet50	32	20	1157s	GPU:V100 RAM:High-RAM
CLAHE+Canny+MobileNet	32	25	996s	GPU:V100 RAM:High-RAM
CLAHE+Canny+MobileNetv2	32	25	567s	GPU:TPU RAM:High-RAM
CLAHE+Canny+VGG19	32	25	4066s	GPU:T4 RAM:High-RAM
CLAHE+Canny+InceptionV3	32	25	555s	GPU:V100 RAM:High-RAM
CLAHE+Canny+VGG16	32	25	776s	GPU:TPU RAM:High-RAM
CLAHE+Canny+ResNet101	32	25	731s	GPU:V100 RAM:High-RAM
Gaussian+Prewitt+CNN	32	20	767s	GPU:V100 RAM:High-RAM
Gaussian+Prewitt+ResNet50	64	25	2335s	GPU:T4 RAM:High-RAM
Gaussian+Prewitt+MobileNet	32	20	966s	GPU:V100 RAM:High-RAM
Gaussian+Prewitt+MobileNetv2	32	25	964s	GPU:V100 RAM:High-RAM
Gaussian+Prewitt+VGG19	32	25	970s	GPU:T4 RAM:High-RAM
Gaussian+Prewitt+InceptionV3	32	25	168s	GPU:V100 RAM:High-RAM
Gaussian+Prewitt+VGG16	32	25	789s	GPU:TPU RAM:High-RAM
Gaussian+Prewitt+ResNet101	32	25	1486s	GPU:TPU RAM:High-RAM
CLAHE+Prewitt+CNN	32	20	1695s	GPU:V100 RAM:High-RAM
CLAHE+Prewitt+ResNet50	32	25	2333s	GPU:T4 RAM:High-RAM
CLAHE+Prewitt+MobileNet	32	25	593s	GPU:TPU RAM:High-RAM
CLAHE+Prewitt+MobileNetv2	32	25	866s	GPU:V100 RAM:High-RAM
CLAHE+Prewitt+VGG19	32	25	1770s	GPU:TPU RAM:High-RAM
CLAHE+Prewitt+InceptionV3	32	25	676s	GPU:V100 RAM:High-RAM
CLAHE+Prewitt+VGG16	32	30	616s	GPU:V100 RAM:High-RAM
CLAHE+Prewitt+ResNet101	32	25	708s	GPU:V100 RAM:High-RAM
LIME with MoblieNetV2	32	25	3560s	GPU:TPU RAM:High-RAM
Grad-CAM with MoblieNetV2	32	25	2414s	GPU:TPU RAM:High-RAM
Grad-CAM with VGG16	32	25	11954s	GPU:TPU RAM:High-RAM
Grad-CAM++with MoblieNetV2	32	20	1509s	GPU:TPU RAM:High-RAM
Grad-CAM++ with VGG16	32	25	11886s	GPU:TPU RAM:High-RAM

where A^k represents the k-th feature map with spatial dimensions $u \times v$.

- Backward Pass (Gradient Computation): The gradients of the class score y^c corresponding to class c are computed with respect to the activations:

$$\frac{\partial y^c}{\partial A_{ij}^k} \tag{7}$$

where A_{ij}^k is the activation at location (i, j) in the k-th feature map.

- Alpha Calculation (Importance Weights): Grad-CAM++ introduces α_{ij}^k, an improved weighting mechanism to assign importance to each spatial location:

$$\alpha_{ij}^k = \frac{\partial^2 y^c}{\partial (A_{ij}^k)^2} \tag{8}$$

These second-order derivatives better capture pixel-level importance in the feature map.

- Weighted Sum of Gradients: Using the improved weights α_{ij}^k, the feature importance w_k^c for each feature map is computed as:

$$w_k^c = \sum_i \sum_j \alpha_{ij}^k \cdot \text{ReLU}\left(\frac{\partial y^c}{\partial A_{ij}^k}\right) \tag{9}$$

This ensures that only positive contributions are considered.

- Heatmap Generation: The final heatmap L^c is computed as:

$$L^c = \text{ReLU}\left(\sum_k w_k^c A^k\right) \tag{10}$$

$$\hat{L}^c = \text{Upsample}(L^c) \tag{11}$$

where $\hat{L}^c$ is the final heatmap, upsampled and overlaid onto the input image.

2) LOCAL INTERPRETABLE MODEL-AGNOSTIC EXPLANATIONS

A method in the area of explainable AI called LIME aims to provide explanations for any machine learning model's predictions that are comprehensible to humans [45]. Because it can be applied to any kind of machine learning model

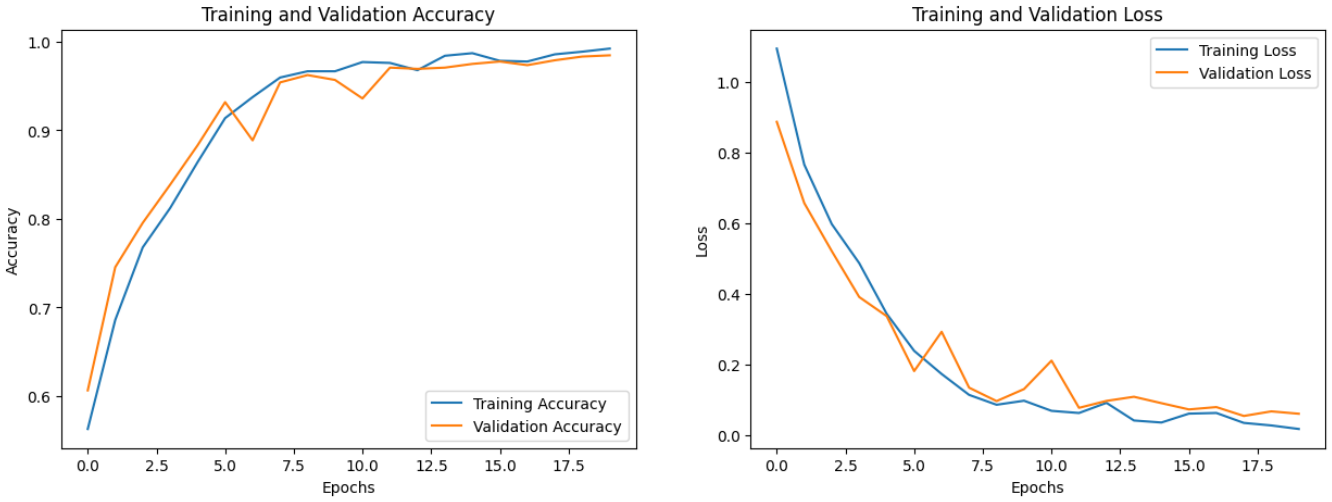


FIGURE 8. Gaussian+Canny+CNN.

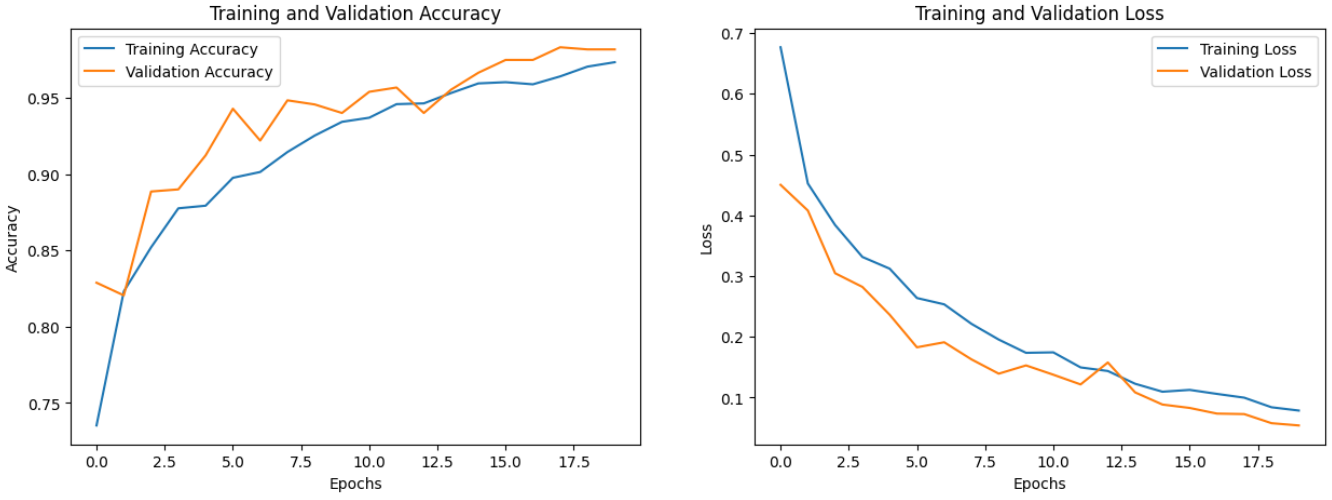


FIGURE 9. Gaussian+Prewitt+MobileNet.

without requiring access to the model's internal workings, it is especially helpful because it is model-agnostic.

The LIME process follows these key steps:

- Perturbation: Given an instance x to be explained, LIME generates a set of perturbed instances X' by slightly modifying its features. The perturbation process ensures diversity while preserving key attributes:

$$X' = \{x'_1, x'_2, \ldots, x'_N\} \tag{12}$$

where N is the number of perturbed instances.

- Prediction: The complex model f is used to generate predictions $f(X')$ for the perturbed instances:

$$Y' = f(X') \tag{13}$$

where Y' represents the predicted outputs for the perturbed samples.

- Weighting: To ensure that perturbed samples closer to the original instance x have greater influence, a similarity function $\pi_x(x')$ assigns weights:

$$\pi_x(x') = e^{-\frac{d(x,x')^2}{\sigma^2}} \tag{14}$$

where $d(x, x')$ is the distance between x and x', and σ controls the locality of the explanation.

- Training an Interpretable Model: A simple interpretable model g, such as a linear regression or decision tree, is trained on the perturbed dataset X', using the weights $\pi_x(x')$ and the predictions Y':

$$g = \arg\min_{g \in G} \sum_{x' \in X'} \pi_x(x')(f(x') - g(x'))^2 \tag{15}$$

where G is the family of interpretable models.

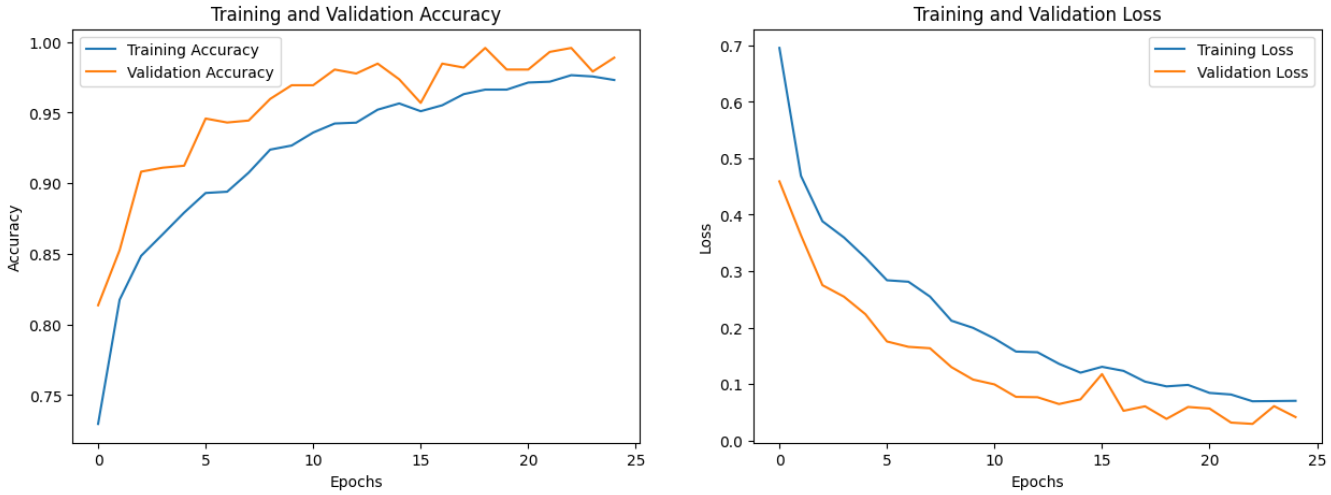


FIGURE 10. Gaussian+Prewitt+MobileNeV2.

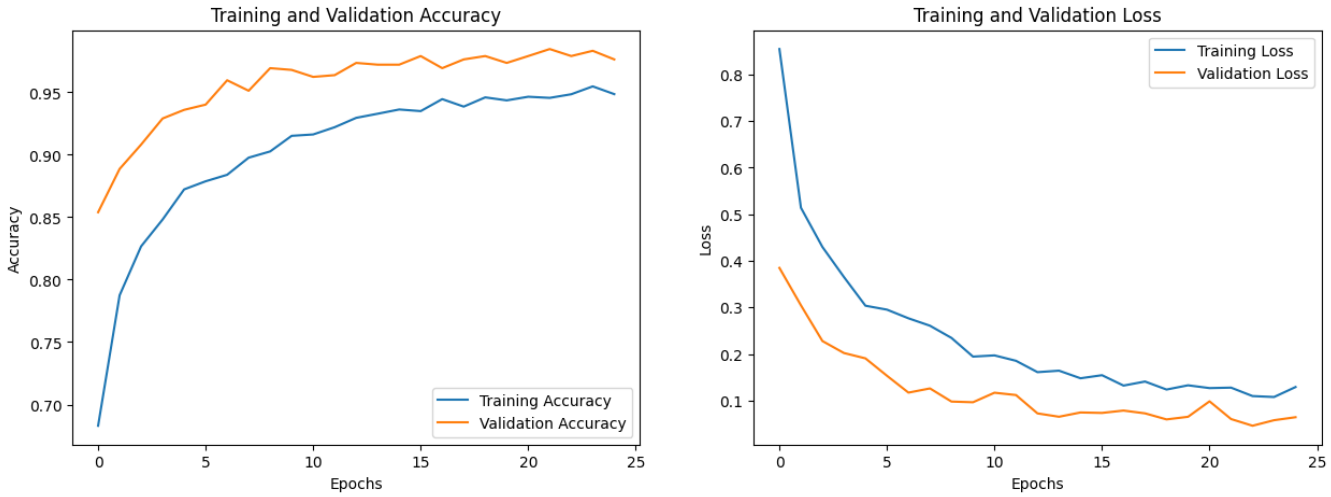


FIGURE 11. CLAHE+Prewitt+ResNet50.

- Generating Explanations: The interpretable model g provides explanations for the original instance x. In the case of a linear model, the feature weights w_i indicate the importance of each feature:

$$g(x) = w_0 + \sum_i w_i x_i \tag{16}$$

where w_i represents the contribution of feature x_i to the prediction.

The following steps summarize the proposed Explainable AI methods:

- Step 1: Collection of Rice Leaf Diseases Images
- Step 2: Resize images to $224 \times 224 \times 3$ dimensions to ensure uniform input for the model.
- Step 3: Classification is performed using deep learning models and measures performance.
- Step 4: This step uses Explainable AI (XAI) techniques to make the decision-making process of a deep learning model more transparent and interpretable, clarifying how it arrived at its classifications (See Figure 7).

IV. EXPERIMENTS

A. PERFORMANCE EVALUATION METRICS

In this work, we employed various performance metrics to assess the efficacy and accuracy of our machine learning model. Different metrics were chosen based on the type of problem and the specific evaluation requirements. The primary classification metrics applied in this study include **Accuracy**, **Precision**, **Recall**, and **F1-Score**.

Equations (17)–(20) provide the mathematical definitions for each of these metrics based on the confusion matrix.

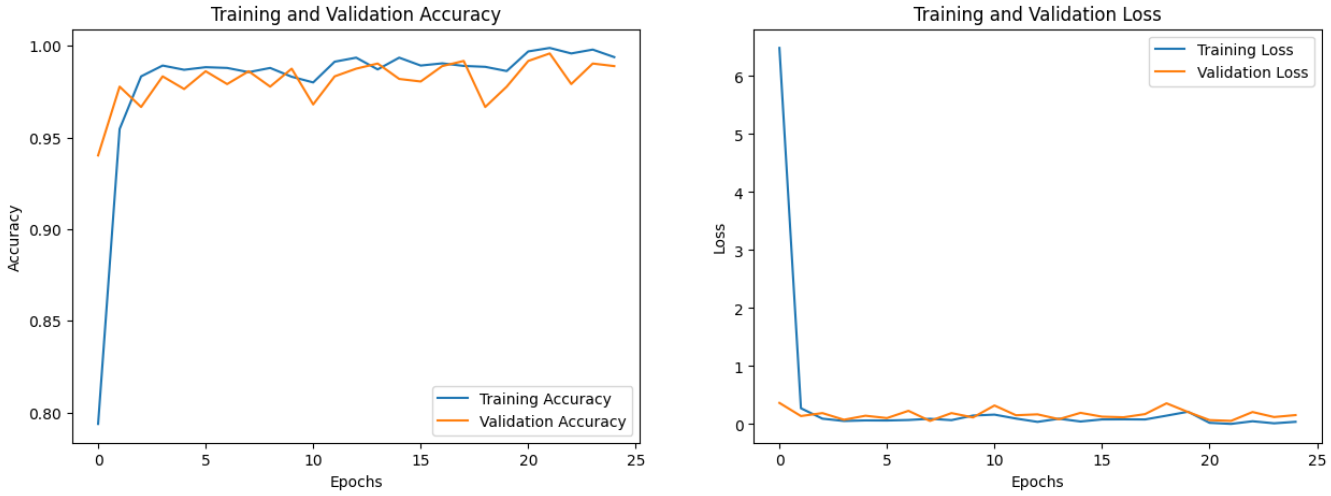


FIGURE 12. CLAHE+Canny+ResNet101.

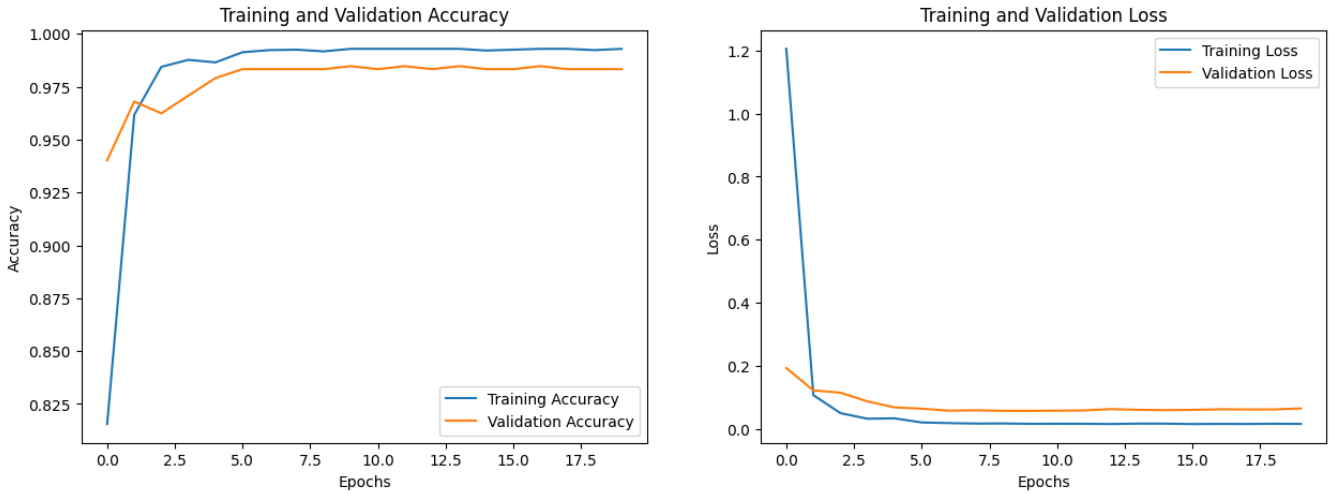


FIGURE 13. Gaussian+Canny+VGG16.

$$\text{Accuracy} = \frac{TP_{\text{Rice Leaf}} + TN_{\text{Rice Leaf}}}{TP_{\text{Rice Leaf}} + FP_{\text{Rice Leaf}} + TN_{\text{Rice Leaf}} + FN_{\text{Rice Leaf}}} \tag{17}$$

$$\text{Precision} = \frac{TP_{\text{Rice Leaf}}}{TP_{\text{Rice Leaf}} + FP_{\text{Rice Leaf}}} \tag{18}$$

$$\text{Recall} = \frac{TP_{\text{Rice Leaf}}}{TP_{\text{Rice Leaf}} + FN_{\text{Rice Leaf}}} \tag{19}$$

$$\text{F1 Score} = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} \tag{20}$$

where $TP_{\text{Rice Leaf}}$ (True Positive), $TN_{\text{Rice Leaf}}$ (True Negative), $FP_{\text{Rice Leaf}}$ (False Positive), and $FN_{\text{Rice Leaf}}$ (False Negative) represent the confusion matrix elements.

In addition to these standard classification metrics [46], we applied explainability-focused metrics—**Feature Coverage**, **Feature Relevance**, and **Consistency Error**—to evaluate the interpretability of the model using Grad-CAM and Grad-CAM++. These metrics offer insight into the reliability and relevance of the highlighted regions in the model's visual explanations, providing an additional layer of validation for model predictions.

B. SETUP

The dataset was divided into training and testing sets, with 80% allocated for training and 20% reserved for testing. The input image size was set to 224×224×3. Training was conducted over 20, 25, and 30 epochs, with a batch size of 64 for disease classification. The study was implemented using Google Colaboratory Pro (Colab Pro), which provided access to TPUs and powerful GPUs, such as A100 and V100. Additionally, the ''High-RAM'' runtime option available in Colab Pro was utilized to support memory-intensive computations. Table 4 shows computational resources during model training.

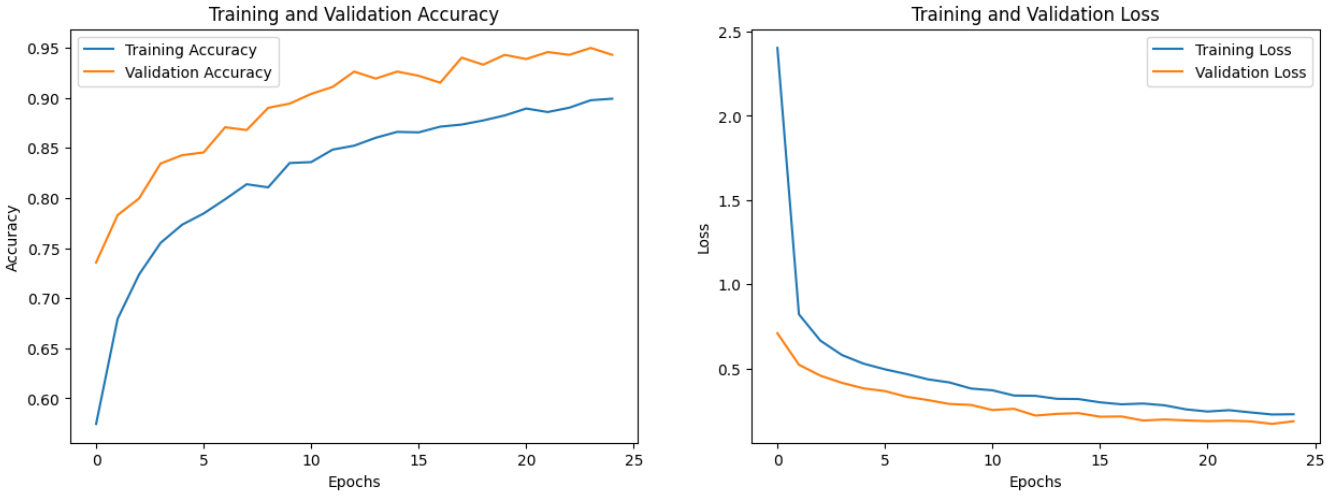


FIGURE 14. CLAHE+Prewitt+VGG19.

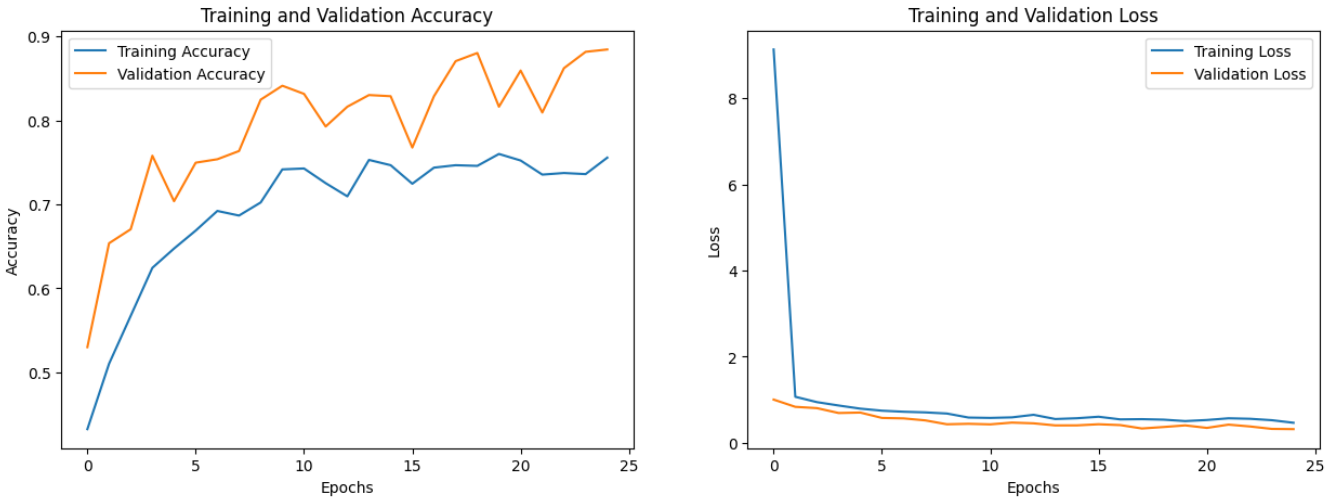


FIGURE 15. Gaussian+Canny+InceptionV3.

The model's learning rate was 0.001, and the Adam optimizer was employed to mitigate overfitting.

The Adam (Adaptive Moment Estimation) optimizer is an advanced gradient-based optimization algorithm that combines the advantages of momentum and adaptive learning rates. It improves convergence speed and stability in deep learning models.

At each iteration, Adam updates the model parameters θ_t using the following steps:

- Compute the Gradients: The gradient g_t of the loss function $L(\theta)$ with respect to the model parameters θ at time step t is calculated as:

$$g_t = \nabla_\theta L(\theta_t) \tag{21}$$

- Compute First and Second Moment Estimates: Adam maintains two moving averages:
  - **First moment estimate** (mean of past gradients, similar to momentum):

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t \tag{22}$$

  - **Second moment estimate** (uncentered variance of past gradients, like RMSprop):

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2 \tag{23}$$

  Here, β_1 and β_2 are decay rates (typically set to 0.9 and 0.999, respectively).
- Bias Correction: Since m_t and v_t are initialized at zero, they are biased towards zero in early iterations. The bias-corrected estimates are:

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t} \tag{24}$$

- Parameter Update: The model parameters are updated using the corrected moment estimates:

$$\theta_t = \theta_{t-1} - \frac{\alpha}{\sqrt{\hat{v}_t} + \epsilon} \hat{m}_t \tag{25}$$

where:

- α is the learning rate.
- ϵ (a small constant, usually 10^{-8}) prevents division by zero.

C. PERFORMANCE CURVE

Training accuracy and training loss Curve presented in Figure 8 to 15 showed that by using the CNN, MobileNet, MobileNetV2, Resnet50, ResNet101, VGG16, VGG19, Inceptionv3 models with difference preprocessing and feature extraction.

D. CONFUSION MATRIX

The confusion matrix is a widely used metric in image classification, helping to assess model performance by differentiating between correctly and incorrectly classified images across various classes [47]. Analyzing the confusion matrices of the recognition results, shown in Figures 16 to 23, reveals that using models like CNN, MobileNet, MobileNetV2, ResNet50, ResNet101, VGG16, VGG19, and Inceptionv3 with different preprocessing and feature extraction techniques yields varying performance outcomes.

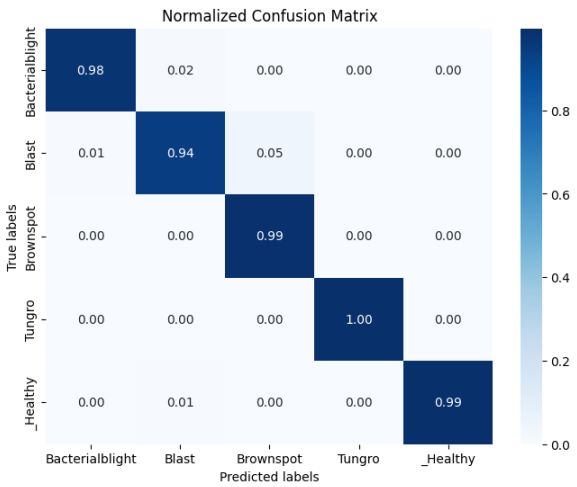


FIGURE 16. Gaussian+Canny+CNN.

E. RESULTS

Table 5 shows that Gaussian blur and Canny edge detection preprocessing was used through all models. ResNet50 is a potent deep learning model that achieves almost flawless classification performance with a performance of 0.99 through all matrices (see Figure 24). VGG16, distinguished by its deep architecture, excels in this preprocessing, attaining superior results on all measures. ResNet50 also

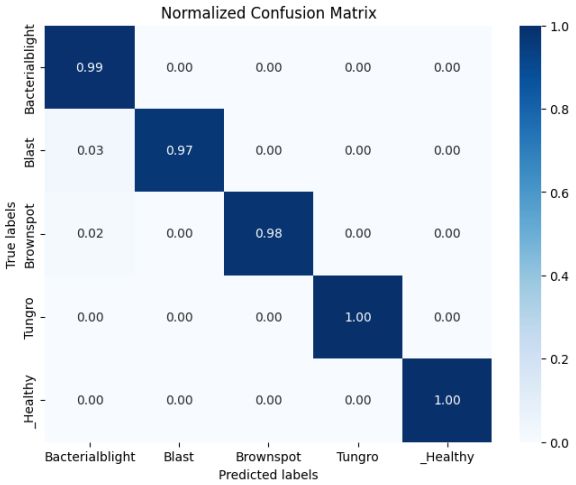


FIGURE 17. Gaussian+Prewitt+MobileNet.

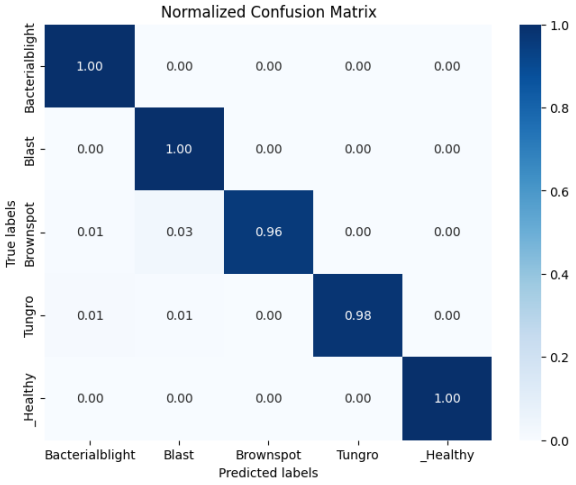


FIGURE 18. Gaussian+Prewitt+MobileNeV2.

maintains a performance of 0.98 through all matrices, demonstrating that CNNs are a good platform for this preprocessing method. MobileNetV2, a fast and efficient model, shows that even lightweight models may achieve excellent accuracy, with a performance of 0.98 through all matrices. ResNet101, with its very large network, provides excellent accuracy and dependability. MobileNet, specifically developed for mobile and embedded vision applications, has reasonable performance but falls short of achieving the exceptional accuracy levels seen by some other models. InceptionV3, renowned for its inception modules designed to enhance computational efficiency, has notably decreased performance. This investigation reveals that although this preprocessing is typically beneficial through many models, its efficacy may vary greatly depending on the unique design of the model. ResNet50 and VGG16 are especially well-suited for this preprocessing approach, earning the greatest performance metrics.

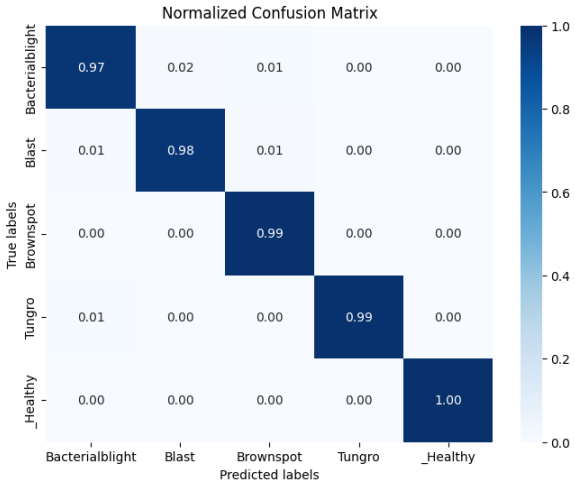


FIGURE 19. CLAHE+Prewitt+ResNet50.

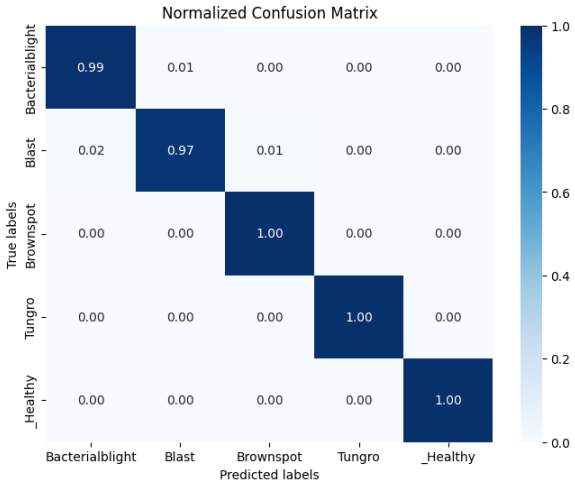


FIGURE 20. CLAHE+Canny+ResNet101.

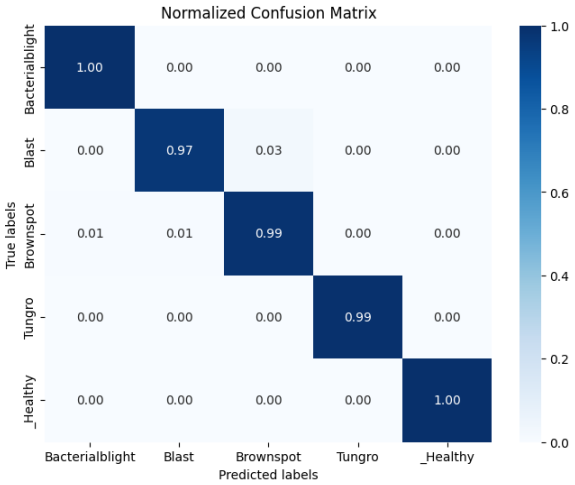


FIGURE 21. Gaussian+Canny+VGG16.

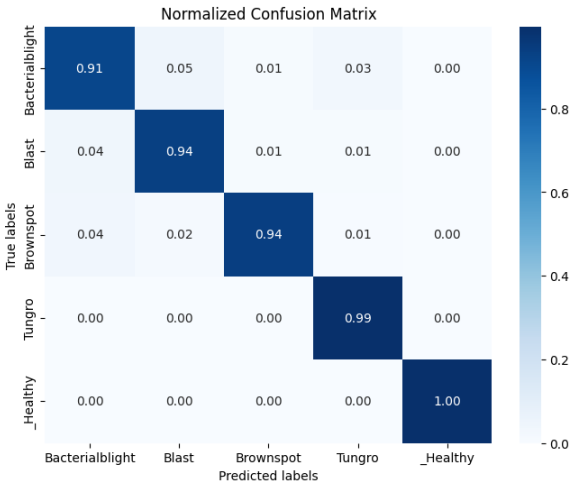


FIGURE 22. CLAHE+Prewitt+VGG19.

Table 6 shows that all models in this study utilized CLAHE and Canny edge detection for preprocessing. ResNet101's deep architecture benefits considerably from this preprocessing, achieving a performance of 0.99 through all matrices, resulting in near-perfect classification results(see Figure 25). ResNet50, renowned for its residual learning capabilities, also performs quite well. The strong performance of a conventional CNN architecture shows the resilience of this preprocessing technique. VGG19, with its deep layers and consistent design, makes good use of the preprocessing. MobileNet, designed for mobile and efficient computation, performs well but not as well as ResNet or VGG models.

TABLE 5. Performance evaluation of gaussian and canny algorithm with each model.

Model	Accuracy	Presion	Recall	F1-Score
CNN	0.98	0.98	0.98	0.98
ResNet50	0.99	0.99	0.99	0.99
MobileNet	0.96	0.96	0.96	0.96
MobileNetV2	0.98	0.98	0.98	0.98
ResNet101	0.99	0.99	0.98	0.98
InceptionV3	0.89	0.89	0.89	0.89
VGG19	0.98	0.98	0.98	0.98
VGG16	0.99	0.99	0.99	0.99

TABLE 6. Performance evaluation of CLAHE and canny algorithm with each model.

Model	Accuracy	Presion	Recall	F1-Score
CNN	0.98	0.98	0.98	0.98
ResNet50	0.98	0.98	0.98	0.98
MobileNet	0.97	0.98	0.97	0.97
MobileNetV2	0.97	0.97	0.97	0.97
ResNet101	0.99	0.99	0.99	0.99
InceptionV3	0.89	0.90	0.89	0.89
VGG19	0.84	0.85	0.84	0.85
VGG16	0.98	0.98	0.98	0.98

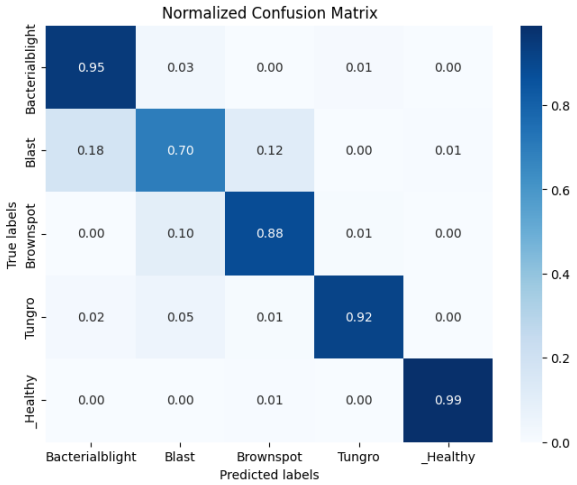


FIGURE 23. Gaussian+Canny+InceptionV3.

MobileNetV2, an enhanced version of MobileNet, performs equally well but still lags behind the best performers. VGG16, despite being deeper than VGG19, does not perform as well, indicating that the extra depth does not compensate for the less successful preprocessing combination. InceptionV3 performs quite well given its intricate architecture designed for computational efficiency, though the preprocessing approach may not be ideal for extracting features at different scales as required by InceptionV3. This analysis shows that while CLAHE and Canny preprocessing are usually successful, the particular model design greatly affects their efficacy. Especially well-suited for this preprocessing method, ResNet101 achieves the best performance metrics.

TABLE 7. Performance evaluation of gaussian and prewitt with each model.

Model	Accuracy	Presion	Recall	F1-Score
CNN	0.98	0.98	0.98	0.98
ResNet50	0.99	0.99	0.99	0.99
MobileNet	0.99	0.99	0.99	0.99
MobileNetV2	0.99	0.99	0.99	0.99
ResNet101	0.99	0.99	0.99	0.99
InceptionV3	0.93	0.93	0.93	0.93
VGG19	0.92	0.95	0.92	0.93
VGG16	0.99	0.99	0.99	0.99

Table 7 shows that the performance evaluation of various models using Gaussian and Prewitt preprocessing algorithms reveals significant insights. The CNN model achieves high performance with an accuracy, precision, recall, and F1 score of 0.98(see Figure 26). ResNet50, MobileNet, MobileNetV2, ResNet101, and VGG16 all attain exceptional performance metrics of 0.99 through all measures, indicating their strong compatibility with the Gaussian and Prewitt preprocessing. These results demonstrate the efficiency of these models in handling the enhanced edge detection and noise reduction provided by the preprocessing techniques. InceptionV3 shows a notably lower performance with scores of 0.93 in all metrics, suggesting that the preprocessing approach may not align well with its architectural needs. VGG19, while performing better than InceptionV3, shows slightly lower metrics compared to the other high-performing models, indicating that the extra depth in VGG19 does not significantly enhance its performance with this preprocessing method. Overall, the evaluation highlights that models like ResNet and VGG16 are particularly well-suited for the Gaussian and Prewitt preprocessing, achieving near-perfect performance metrics.

TABLE 8. Performance evaluation of CLAHE and prewitt with each model.

Model	Accuracy	Presion	Recall	F1-Score
CNN	0.99	0.99	0.99	0.99
ResNet50	0.99	0.99	0.99	0.99
MobileNet	0.97	0.97	0.97	0.97
MobileNetV2	0.98	0.98	0.98	0.98
ResNet101	0.99	0.99	0.99	0.99
InceptionV3	0.86	0.88	0.87	0.86
VGG19	0.95	0.95	0.96	0.95
VGG16	0.99	0.99	0.99	0.99

Table 8 shows that the performance evaluation of various models using CLAHE and Prewitt preprocessing algorithms demonstrates the effectiveness of these techniques. The CNN, ResNet50, ResNet101, and VGG16 models achieve outstanding performance metrics of 0.99 through all measures (accuracy, precision, recall, and F1 score), highlighting their strong compatibility with CLAHE and Prewitt preprocessing(see Figure 27). MobileNet and MobileNetV2 also perform well, with MobileNet achieving scores of 0.97 and MobileNetV2 achieving 0.98 through all metrics, reflecting their efficiency, though slightly lower than the top performers. InceptionV3 shows a significantly lower performance, with scores of 0.86 in accuracy and F1 score, 0.88 in precision, and 0.87 in recall, suggesting that its architectural design may not align well with this preprocessing approach. VGG19, similar to its performance with other preprocessing methods, shows good but not exceptional performance with scores of 0.92 through all metrics. Overall, the evaluation indicates that CLAHE and Prewitt preprocessing techniques are highly effective, particularly for models like CNN, ResNet50, ResNet101, and VGG16, which achieve near-perfect performance.

F. EXPLORING EXPLAINABLE AI METHODS

The performance evaluation table 9 shows, the effectiveness of different visualization [48] techniques, specifically LIME and various forms of Grad-CAM, with MobileNetV2 and VGG16 models. through all evaluations, including LIME with MobileNetV2, Grad-CAM++ with VGG16, Grad-CAM+ MobileNetV2, and Grad-CAM++ VGG16, all models consistently achieve high scores of 0.99 for accuracy,

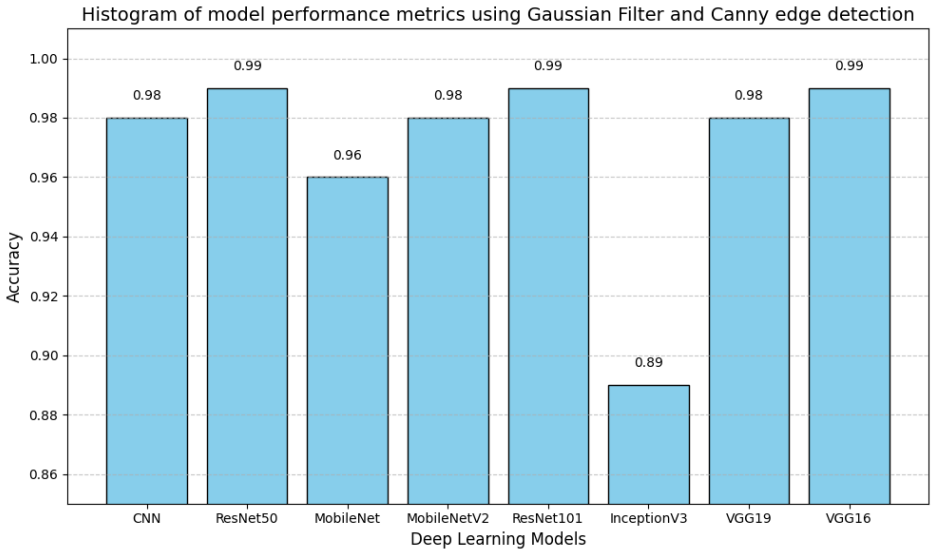


FIGURE 24. Model performance metrics using gaussian filter and canny edge detection.

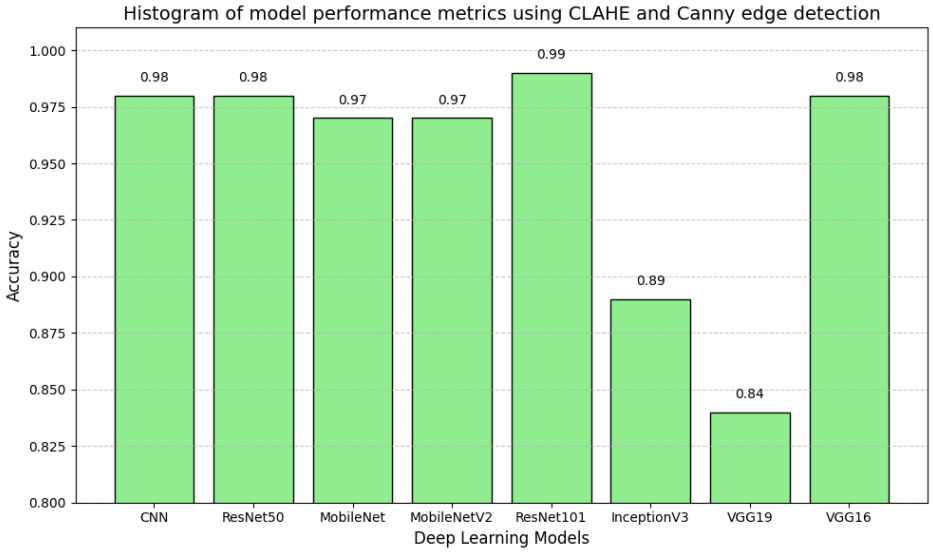


FIGURE 25. Model performance metrics using CLAHE and canny edge detection.

TABLE 9. Performance evaluation of explainable AI method with each model.

Model	Accuracy	Presion	Recall	F1-Score
Grad-CAM++ with MobileNetV2	0.99	0.99	0.99	0.99
Grad-CAM++ with VGG16	0.99	0.99	0.99	0.99
Grad-CAM+ MobileNetV2	0.99	0.99	0.99	0.99
Grad-CAM+ VGG16	0.99	0.99	0.99	0.99
LIME+ MobileNetV2	0.99	0.99	0.99	0.99

precision, recall, and F1 score. LIME provides detailed and accurate visual explanations of model predictions, while Grad-CAM techniques offer insights into which parts of the image are most influential for the model's decision-making process.

This evaluation underscores the utility of explainable AI techniques like LIME, Grad-CAM and Grad-CAM++ in improving the transparency and trustworthiness of deep learning models, particularly in applications where understanding model predictions is critical. Figures 28, 29,

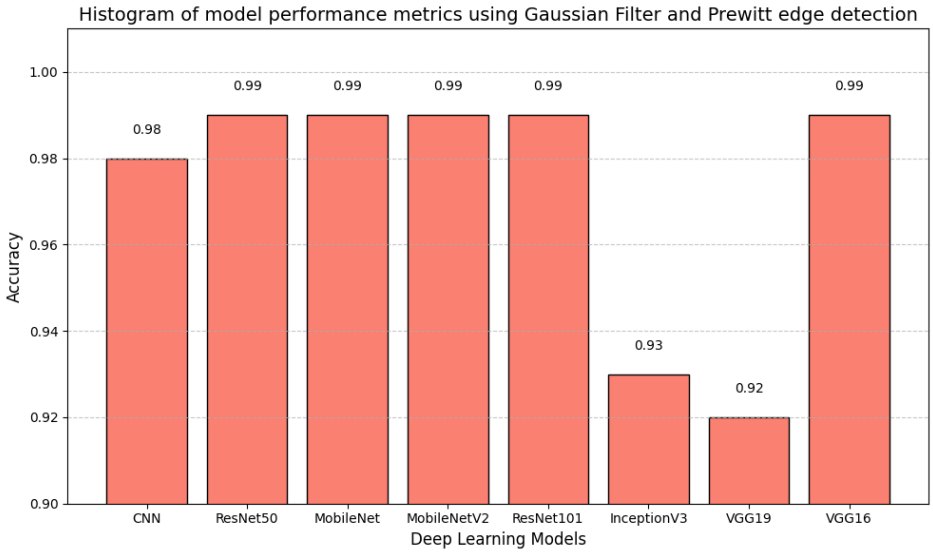


FIGURE 26. Model performance metrics using gaussian filter and prewitt edge detection.

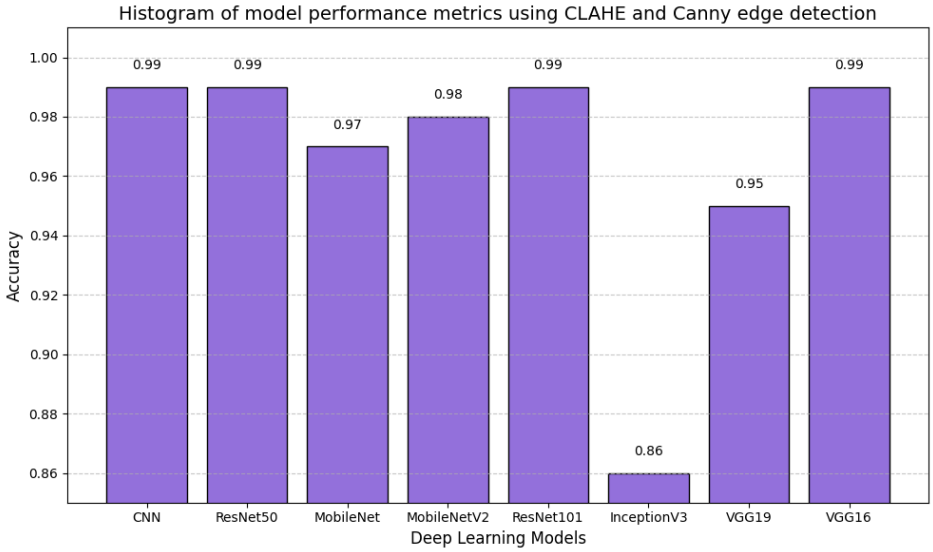


FIGURE 27. Model performance metrics using CLAHE and prewitt edge detection.

TABLE 10. LIME evaluation metrics for MobileNetV2.

Evaluation Metric	Score	Explanation
Faithfulness (Fidelity)	0.97	High fidelity, meaning the surrogate model explains the original model well.
Ground Truth Mask Overlap	0.95	95% overlap between LIME's highlighted regions and the ground truth mask.
Perturbation Consistency	0.90	LIME explanations remain consistent across perturbed versions of the image.
Stability	0.95	Explanations are 95% consistent across runs.

30, and 31, along with Table 33, present the evaluation metrics for Grad-CAM and Grad-CAM++ applied to MobileNetV2 and VGG16. Similarly, Figure 32 and Table 10 present the evaluation metrics for LIME applied to MobileNetV2.

G. COMPARISON WITH PREVIOUS STUDIES

Table 11 presents a performance comparison between our proposed study and existing works on rice leaf disease detection. The accuracy of our model is 99%, outperforming all previous approaches.

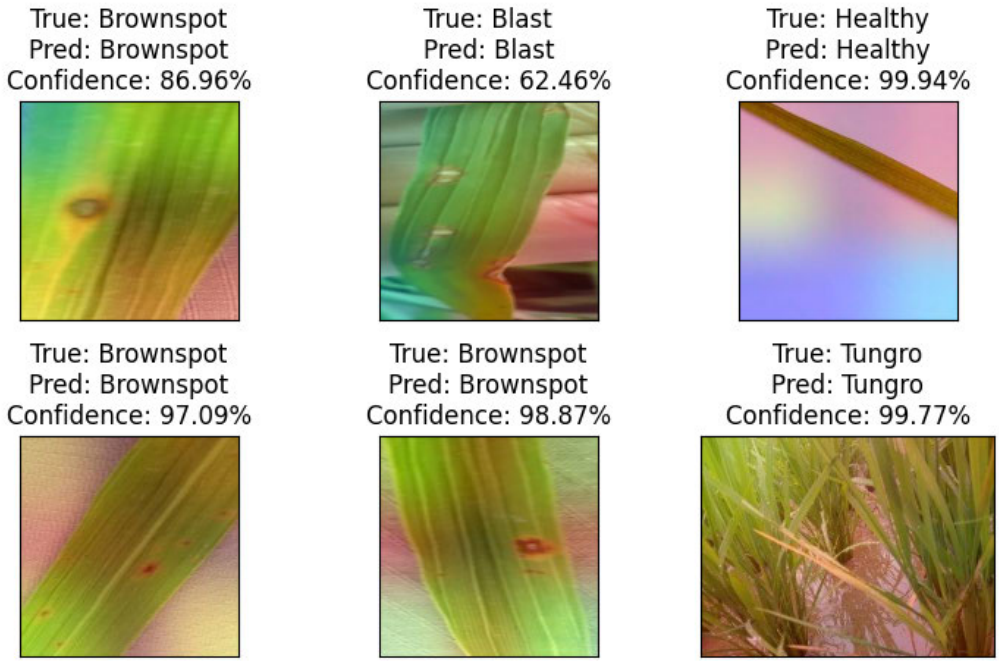


FIGURE 28. MobileNetV2 with Grad-CAM.

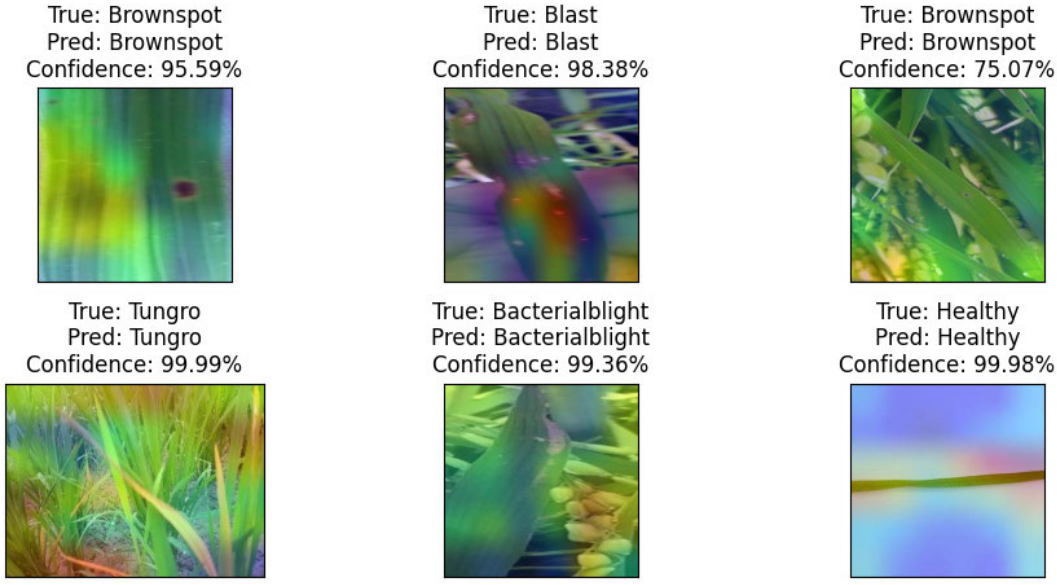


FIGURE 29. VGG16 with Grad-CAM.

Tejaswini et al. [2] achieved a lower accuracy of 78%, indicating room for improvement in their method. Zeng et al. [4] and Aggarwal et al. [17] improved upon this, reaching 95.57% and 94%, respectively. Several recent studies, such as those by Sudhesh et al. [16], Rajpoot et al. [19], and Chaudhari et al. [26], reported high accuracy ranging from 94% to 97.3%.

More recent advancements, including the works of Stephen et al. (2024) [29] and Ji et al. (2025) [52], demonstrated strong performances, achieving 98.7% and 97.5%, respectively. However, our proposed method surpasses these results with a 99% accuracy, highlighting its effectiveness.

The improvement in accuracy can be attributed to our deep learning approach, which integrates Explain-

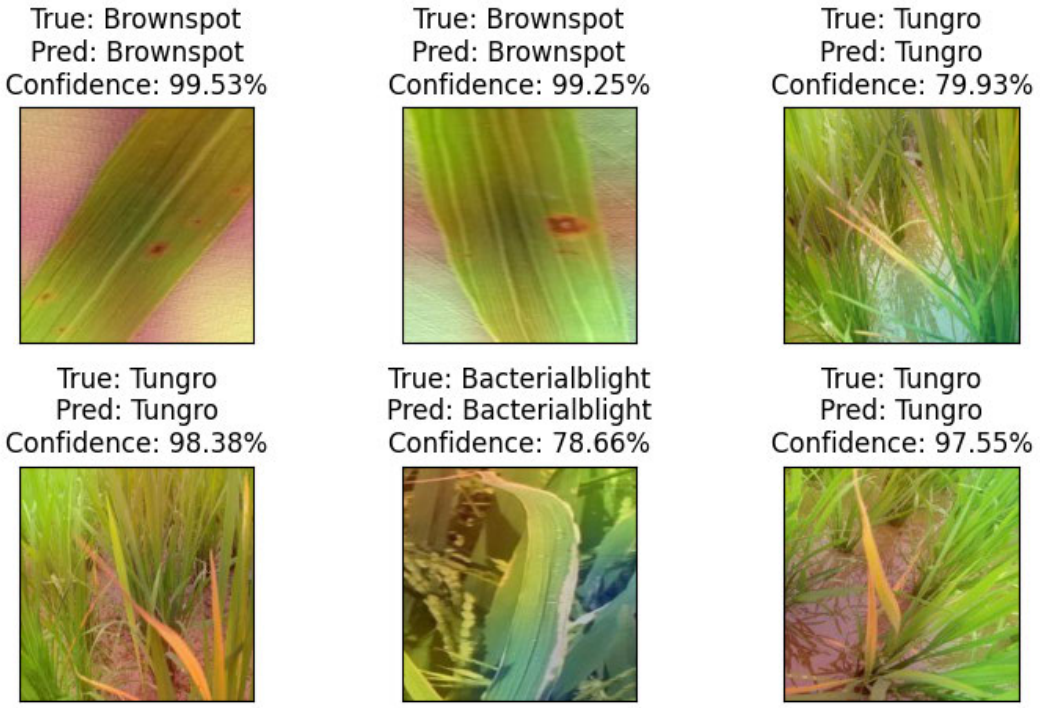


FIGURE 30. MobileNetV2 with Grad-CAM++.

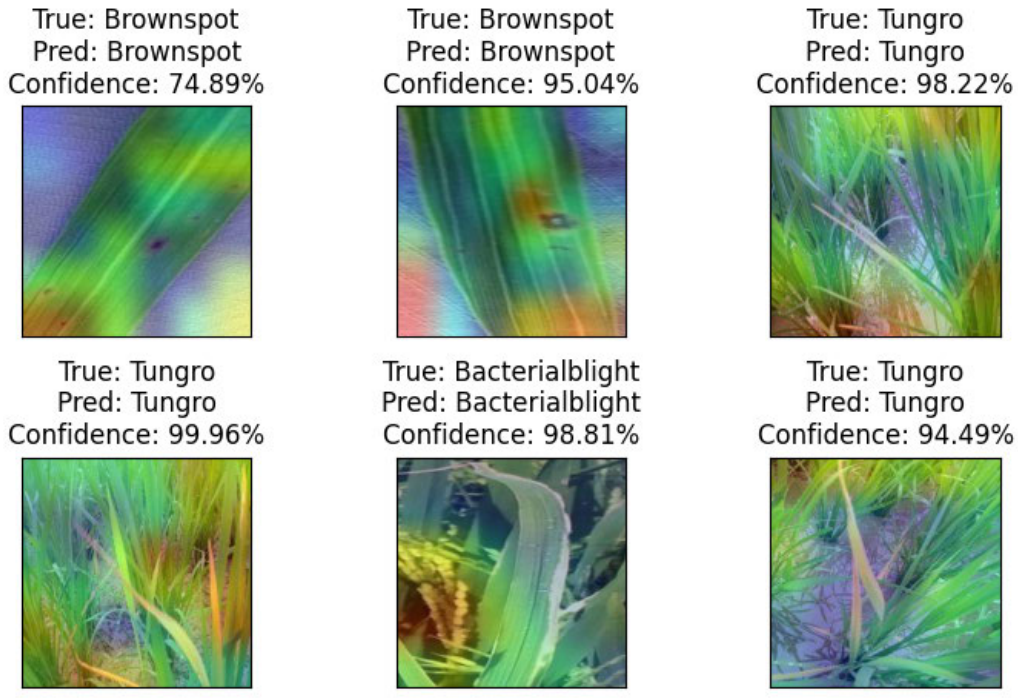


FIGURE 31. VGG16 with Grad-CAM++.

able AI (XAI) techniques for precise disease classification. By leveraging advanced architectures and feature extraction techniques, our model enhances interpretability and robustness. This demonstrates the superiority of our approach in ensuring reliable and accurate rice leaf disease detection.

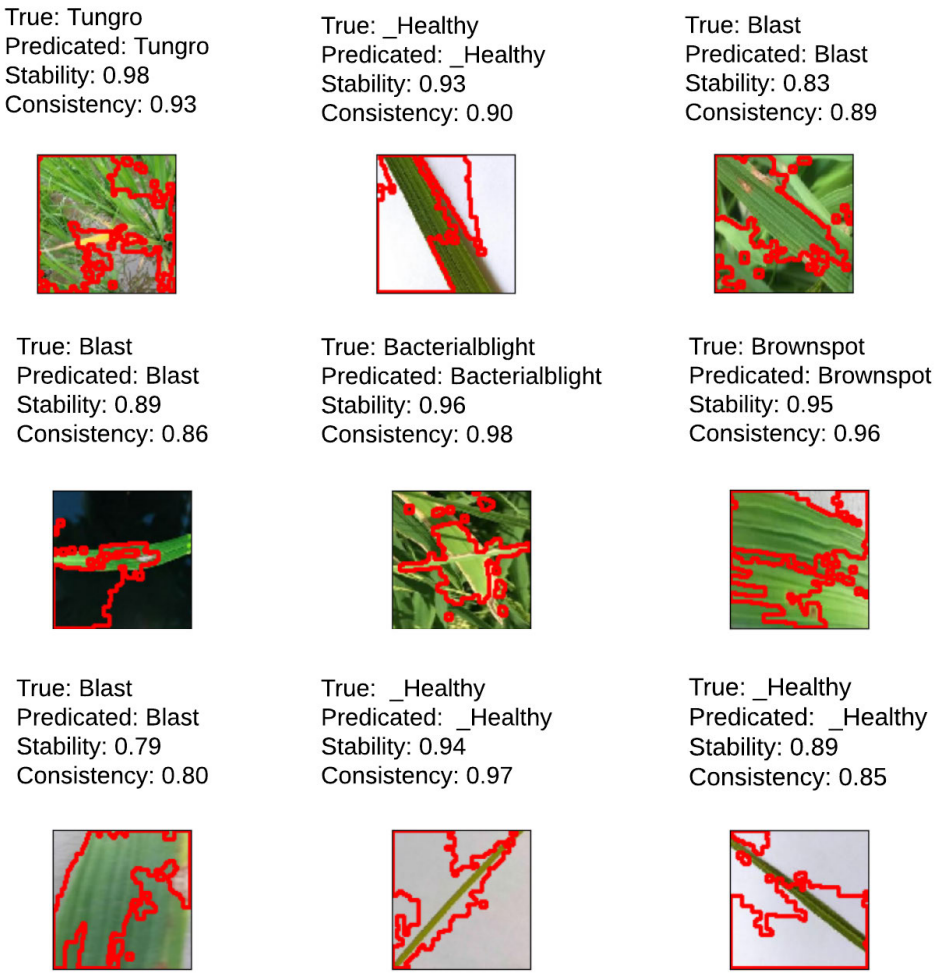


FIGURE 32. MobileNetV2 with LIME.

Evaluation Metric	Grad-CAM	Grad-CAM++	Explanation
Ground Truth Mask Overlap	95%	98%	Percentage of overlap between the ground truth mask and the highlighted region.
Feature Coverage	0.95	0.98	Proportion of the image covered by the highlighted features in the explanation.
Relevant Activation	94%	99%	Percentage of activation in relevant areas.
Feature Relevance	0.94	0.99	Relevance of features in the explanation, corresponding to the model's decision.
Similarity (Mean Absolute Error)	1.1	0.89	Mean absolute error between the predicted and actual class.
Consistency Error	1.1	0.89	Error in consistency when input is perturbed or modified.

FIGURE 33. Evaluation metrics for Grad-CAM and Grad-CAM++ for MobileNetV2.

TABLE 11. Performance comparison with existing methods.

Study	Performance (Accuracy)
Tejaswini et al. (2022) [2]	78%
Zeng et al. (2023) [4]	95.57%
Aggarwal et al. (2023) [17]	94%
Sudhesh et al. (2023) [16]	94%
Rajpoot et al. (2023) [19]	97.3%
Chaudhari et al. (2023) [26]	97%
Rezki et al. (2023) [49]	97.23%
Stephen et al. (2024) [29]	98.7%
Trinh et al. (2024) [50]	89.9%
Kaur et al. (2024): [30]	93.3%
Kulkarni et al. (2024) [31]	95%
Dai et al. (2024) [32]	83.1%
Kiratiratanapruk et al. (2025) [51]	94.20%
Ji et al. (2025) [52]	97.5%
Our Study	**99%**

V. CONCLUSION

This study presents several pretrained models to detect four types of rice leaf diseases, along with healthy images, with the goal of optimizing computational efficiency. Image recognition and classification for identifying and categorizing leaf diseases is a crucial research area that significantly contributes to the agricultural economy. For image preprocessing, the study employs Gaussian filtering and CLAHE to enhance contrast, while Canny and Prewitt edge detection methods are utilized for feature extraction. The results indicate that the best-performing models, such as VGG16 and ResNet101, both achieved 99% accuracy. Additionally, the integration of visualization techniques like Grad-CAM, Grad-CAM++, and LIME further improved the

interpretability and practical application of the deep learning models. Despite these promising results, the study is limited by the relatively small dataset, which may affect the model's ability to generalize in real-world scenarios. While the model demonstrated strong performance under constrained data conditions, broader environmental variability is essential for reliable deployment. To address this, we plan to expand the dataset by collecting more diverse samples across different field conditions and geographic regions, in collaboration with agricultural experts. In future work, we will incorporate data augmentation, generative image modeling, and domain adaptation to reduce overfitting and enhance generalization capability.

As an additional direction, We will explore frequency domain filtering methods, like Butterworth filtering and Discrete Wavelet Transform (DWT), to evaluate their possible advantages in improving feature extraction and noise reduction for higher illness classification accuracy. These methods provide multi-resolution analysis by capturing both frequency and spatial information, which is advantageous for detecting subtle disease patterns in complex backgrounds.

DATA AVAILABILITY STATEMENT

No supplementary data was utilized in the research described in this article.

Source code of the associated data set to github: https://github.com/sharmin133/Rice-Leaf

ACKNOWLEDGMENT

The authors utilized an AI tool to assist with grammar checking.

REFERENCES

[1] T. Mahmud, N. Datta, R. Chakma, U. Kanti Das, M. T. Aziz, M. Islam, A. H. M. Salimullah, M. S. Hossain, and K. Andersson, "An approach for crop prediction in agriculture: Integrating genetic algorithms and machine learning," *IEEE Access*, vol. 12, pp. 173583–173598, 2024.

[2] P. Tejaswini, P. Singh, M. Ramchandani, Y. K. Rathore, and R. R. Janghel, "Rice leaf disease classification using CNN," *IOP Conf. Ser., Earth Environ. Sci.*, vol. 1032, no. 1, Jun. 2022, Art. no. 012017.

[3] C. G. Simhadri and H. K. Kondaveeti, "Automatic recognition of Rice leaf diseases using transfer learning," *Agronomy*, vol. 13, no. 4, p. 961, Mar. 2023.

[4] N. Zeng, G. Gong, G. Zhou, and C. Hu, "An accurate classification of Rice diseases based on ICAI-V4," *Plants*, vol. 12, no. 11, p. 2225, Jun. 2023.

[5] M. H. Imam, N. Nahar, R. Bhowmik, S. B. S. Omit, T. Mahmud, M. S. Hossain, and K. Andersson, "A transfer learning-based framework: MobileNet-SVM for efficient tomato leaf disease classification," in *Proc. 6th Int. Conf. Electr. Eng. Inf. Commun. Technol. (ICEEICT)*, May 2024, pp. 693–698.

[6] T. Mahmud, K. Barua, A. Barua, N. Basnin, S. Das, M. S. Hossain, and K. Andersson, "Explainable AI for tomato leaf disease detection: Insights into model interpretability," in *Proc. 26th Int. Conf. Comput. Inf. Technol. (ICCIT)*, Dec. 2023, pp. 1–6.

[7] R. P. Narmadha, N. Sengottaiyan, and R. J. Kavitha, "Deep transfer learning based Rice plant disease detection model," *Intell. Autom. Soft Comput.*, vol. 31, no. 2, pp. 1257–1271, 2022.

[8] M. H. Ali, T. Mahmud, M. T. Aziz, M. F. B. A. Aziz, M. S. Hossain, and K. Andersson, "Leveraging transfer learning for efficient classification of coffee leaf diseases," in *Proc. 15th Int. Conf. Comput. Commun. Netw. Technol. (ICCCNT)*, Jun. 2024, pp. 1–6.

[9] P. Dey, T. Mahmud, S. R. Nahar, M. S. Hossain, and K. Andersson, "Plant disease detection in precision agriculture: Deep learning approaches," in *Proc. 2nd Int. Conf. Intell. Data Commun. Technol. Internet Things (IDCIoT)*, Jan. 2024, pp. 661–667.

[10] T. Rahman, T. Mahmud, M. M. Setara, S. Roy, M. S. Hossain, and K. Andersson, "Application of deep learning for detecting Rice leaf diseases in jhum cultivation," in *Proc. 15th Int. Conf. Comput. Commun. Netw. Technol. (ICCCNT)*, Jun. 2024, pp. 1–6.

[11] Y. Wei, Z. Wang, and X. Qiao, "Rice disease recognition and feature visualization using a convolutional neural network," in *Proc. 11th Int. Conf. Comput. Pattern Recognit.*, Nov. 2022, pp. 20–25.

[12] T. Akter, T. Mahmud, R. Chakma, N. Datta, M. S. Hossain, and K. Andersson, "IoT-based precision agriculture monitoring system: Enhancing agricultural efficiency," in *Proc. 2nd Int. Conf. Inventive Comput. Informat. (ICICI)*, Jun. 2024, pp. 749–754.

[13] P. K. Sethy, N. K. Barpanda, A. K. Rath, and S. K. Behera, "Deep feature based Rice leaf disease identification using support vector machine," *Comput. Electron. Agricult.*, vol. 175, Aug. 2020, Art. no. 105527.

[14] L. Feng, B. Wu, Y. He, and C. Zhang, "Hyperspectral imaging combined with deep transfer learning for Rice disease detection," *Frontiers Plant Sci.*, vol. 12, Sep. 2021, Art. no. 693521.

[15] P. Mekha and N. Teeyasuksaet, "Image classification of Rice leaf diseases using random forest algorithm," in *Proc. Joint Int. Conf. Digit. Arts, Media Technol. ECTI Northern Sect. Conf. Electr., Electron., Comput. Telecommun. Eng.*, Mar. 2021, pp. 165–169.

[16] K. Sudhesh, V. Sowmya, S. Kurian, and O. Sikha, "AI based Rice leaf disease identification enhanced by dynamic mode decomposition," *Eng. Appl. Artif. Intell.*, vol. 120, Apr. 2023, Art. no. 105836.

[17] M. Aggarwal, V. Khullar, N. Goyal, A. Singh, A. Tolba, E. B. Thompson, and S. Kumar, "Pre-trained deep neural network-based features selection supported machine learning for Rice leaf disease classification," *Agriculture*, vol. 13, no. 5, p. 936, Apr. 2023.

[18] M. Shobana, S. Sneha, V. Shaini, and J. Sreemathy, "Identification of paddy disease using image processing," in *Proc. 2nd Int. Conf. Intell. Data Commun. Technol. Internet Things (IDCIoT)*, Jan. 2024, pp. 739–744.

[19] V. Rajpoot, A. Tiwari, and A. S. Jalal, "Automatic early detection of Rice leaf diseases using hybrid deep learning and machine learning methods," *Multimedia Tools Appl.*, vol. 82, no. 23, pp. 36091–36117, Sep. 2023.

[20] S. Sagar and J. Singh, "An experimental study of tomato viral leaf diseases detection using machine learning classification techniques," *Bull. Electr. Eng. Informat.*, vol. 12, no. 1, pp. 451–461, Feb. 2023.

[21] J. A. Pandian, S. N. Nisha, K. Kanchanadevi, A. K. Pandey, and S. K. Rima, "Grey blight disease detection on tea leaves using improved deep convolutional neural network," *Comput. Intell. Neurosci.*, vol. 2023, no. 1, Jan. 2023, Art. no. 7876302.

[22] J. Pasalkar, G. Gorde, C. More, S. Memane, and V. Gaikwad, "Potato leaf disease detection using machine learning," *Current Agricult. Res. J.*, vol. 11, no. 3, pp. 949–954, Jan. 2024.

[23] L. Song, B. Haoran, L. Fengmei, D. Wang, Y. Zheng, Q. Jiang, and S. Fengbo, "An apple leaf disease identification model for safeguarding apple food safety," *Food Sci. Technol.*, vol. 43, Jan. 2023, Art. no. 104322.

[24] M. T. Ahad, Y. Li, B. Song, and T. Bhuiyan, "Comparison of CNN-based deep learning architectures for Rice diseases classification," *Artif. Intell. Agricult.*, vol. 9, pp. 22–35, Sep. 2023.

[25] A. Junaidi, Q. Diao, W. H. Chan, and S. Z. M. Hashim, "Enhancing Rice leaf disease classification: A combined algorithm approach for improved accuracy and robustness," in *Proc. Int. Conf. Electron.* Singapore: Springer, 2024, pp. 185–203.

[26] D. J. Chaudhari and K. Malathi, "Detection and prediction of Rice leaf disease using a hybrid CNN-SVM model," *Opt. Memory Neural Netw.*, vol. 32, no. 1, pp. 39–57, Mar. 2023.

[27] U. Bhimavarapu, "Prediction and classification of Rice leaves using the improved PSO clustering and improved CNN," *Multimedia Tools Appl.*, vol. 82, no. 14, pp. 21701–21714, Jun. 2023.

[28] P. K. Mannepalli, A. Pathre, G. Chhabra, P. A. Ujjainkar, and S. Wanjari, "Diagnosis of bacterial leaf blight, leaf smut, and Brown spot in Rice leafs using VGG16," *Proc. Comput. Sci.*, vol. 235, pp. 193–200, May 2024.

[29] A. Stephen, A. Punitha, and A. Chandrasekar, "Optimal deep generative adversarial network and convolutional neural network for Rice leaf disease prediction," *Vis. Comput.*, vol. 40, no. 2, pp. 919–936, Feb. 2024.

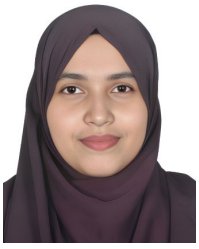
[30] A. Kaur, K. Guleria, and N. K. Trivedi, "A deep learning-based model for biotic Rice leaf disease detection," *Multimedia Tools Appl.*, vol. 83, no. 36, pp. 83583–83609, Mar. 2024.

[31] P. Kulkarni and D. S. Shastri, "Rice leaf diseases detection using machine learning," *J. Sci. Res. Technol.*, vol. 1, pp. 17–22, Jan. 2024.

[32] W. Dai, W. Zhu, G. Zhou, G. Liu, J. Xu, H. Zhou, Y. Hu, Z. Liu, J. Li, and L. Li, "AISOA-SSformer: An effective image segmentation method for Rice leaf disease based on the transformer architecture," *Plant Phenomics*, vol. 6, p. 0218, Aug. 2024.
[33] R. Mukherjee, A. Ghosh, C. Chakraborty, J. N. De, and D. P. Mishra, "Rice leaf disease identification and classification using machine learning techniques: A comprehensive review," *Eng. Appl. Artif. Intell.*, vol. 139, Jan. 2025, Art. no. 109639.
[34] S. Phadikar, J. Sil, and A. K. Das, "Rice diseases classification using feature selection and rule generation techniques," *Comput. Electron. Agricult.*, vol. 90, pp. 76–85, Jan. 2013.
[35] H. Seddik, "A new family of Gaussian filters with adaptive lobe location and smoothing strength for efficient image restoration," *EURASIP J. Adv. Signal Process.*, vol. 2014, no. 1, pp. 1–11, Dec. 2014.
[36] A. Mishra, "Contrast limited adaptive histogram equalization (CLAHE) approach for enhancement of the microstructures of friction stir welded joints," 2021, *arXiv:2109.00886*.
[37] E. Akbari Sekehravani, E. Babulak, and M. Masoodi, "Implementing Canny edge detection algorithm for noisy image," *Bull. Electr. Eng. Informat.*, vol. 9, no. 4, pp. 1404–1410, Aug. 2020.
[38] S. Balochian and H. Baloochian, "Edge detection on noisy images using Prewitt operator and fractional order differentiation," *Multimedia Tools Appl.*, vol. 81, no. 7, pp. 9759–9770, Mar. 2022.
[39] M. X. Morris, A. Rajesh, M. Asaad, A. Hassan, R. Saadoun, and C. E. Butler, "Deep learning applications in surgery: Current uses and future directions," *Amer. Surgeon*, vol. 89, no. 1, pp. 36–42, Jan. 2023.
[40] K. Barua, T. Mahmud, A. Barua, N. Sharmen, N. Basnin, D. Islam, M. S. Hossain, K. Andersson, and S. Hossain, "Explainable AI-based humerus fracture detection and classification from X-ray images," in *Proc. 26th Int. Conf. Comput. Inf. Technol. (ICCIT)*, Dec. 2023, pp. 1–6.
[41] M. M. Rahman, Z. Yan, M. T. Aziz, M. A. B. Siddick, T. Truong, M. M. Sharif, N. Datta, T. Mahmud, R. D. C. Pecho, and S. M. Farid, "Explainable deep transfer learning framework for Rice leaf disease diagnosis and classification," *Int. J. Adv. Comput. Sci. Appl.*, vol. 15, no. 12, pp. 862–884, 2024.
[42] M. M. Musthafa, T. R. Mahesh, and S. Guluwadi, "Enhancing brain tumor detection in MRI images through explainable AI using grad-CAM with ResNet 50," *BMC Med. Imag.*, vol. 24, no. 1, p. 107, May 2024.
[43] T. Mahmud, K. Barua, S. U. Habiba, N. Sharmen, M. S. Hossain, and K. Andersson, "An explainable AI paradigm for Alzheimer's diagnosis using deep transfer learning," *Diagnostics*, vol. 14, no. 3, p. 345, Feb. 2024.
[44] A. Chattopadhay, A. Sarkar, P. Howlader, and V. N. Balasubramanian, "Grad-CAM++: Generalized gradient-based visual explanations for deep convolutional networks," in *Proc. IEEE Winter Conf. Appl. Comput. Vis. (WACV)*, Mar. 2018, pp. 839–847.
[45] V. S. S. V. Chivukula, G. Anuradha, S. N. C. Dhanekula, and N. G. Kothagundla, "Rice crop disease detection using explainable AI," in *Proc. Global Conf. Inf. Technol. Commun. (GCITC)*, Dec. 2023, pp. 1–8.
[46] M. Bhagat and D. Kumar, "Performance evaluation of PCA based reduced features of leaf images extracted by DWT using random forest and XGBoost classifier," *Multimedia Tools Appl.*, vol. 82, no. 17, pp. 26225–26254, Jul. 2023.
[47] J. Li, H. Sun, and J. Li, "Beyond confusion matrix: Learning from multiple annotators with awareness of instance features," *Mach. Learn.*, vol. 112, no. 3, pp. 1053–1075, Mar. 2023.
[48] Y. Shen, B. Sheng, R. Fang, H. Li, L. Dai, S. Stolte, J. Qin, W. Jia, and D. Shen, "Domain-invariant interpretable fundus image quality assessment," *Med. Image Anal.*, vol. 61, Apr. 2020, Art. no. 101654.
[49] N. G. Rezk, E. E.-D. Hemdan, A.-F. Attia, A. El-Sayed, and M. A. El-Rashidy, "An efficient IoT based framework for detecting Rice disease in smart farming system," *Multimedia Tools Appl.*, vol. 82, no. 29, pp. 45259–45292, Dec. 2023.
[50] D. C. Trinh, A. T. Mac, K. G. Dang, H. T. Nguyen, H. T. Nguyen, and T. D. Bui, "Alpha-EIOU-YOLOv8: An improved algorithm for Rice leaf disease detection," *AgriEngineering*, vol. 6, no. 1, pp. 302–317, Feb. 2024.
[51] K. Kiratiratanapruk, P. Temniranrat, W. Sinthupinyo, S. Marukatat, and S. Patarapuwadol, "Automatic detection of Rice disease in images of various leaf sizes," *IET Image Process.*, vol. 19, no. 1, p. 13301, Jan. 2025.
[52] X. Ji, J. Xue, J. Shi, W. Wang, X. Zhang, Z. Wang, W. Lu, J. Liu, Y. V. Fu, and N. Xu, "Noninvasive Raman spectroscopy for the detection of Rice bacterial leaf blight and bacterial leaf streak," *Talanta*, vol. 282, Jan. 2025, Art. no. 126962.



TANJIM MAHMUD (Member, IEEE) received the B.Sc. degree in computer science and engineering and the M.Sc. degree in engineering from the University of Chittagong, Bangladesh, and the Ph.D. degree in engineering with a specialization in natural language processing from the Kitami Institute of Technology, Japan. He is currently an Assistant Professor with the Department of Computer Science and Engineering, Rangamati Science and Technology University, Bangladesh. His research interests include artificial intelligence, natural language processing, AI applications in healthcare and agriculture, machine learning, and image processing.



MST. SHARMIN AKTER is currently pursuing the degree in computer science and engineering with Port City International University, Chattogram, Bangladesh. Her research interests include machine learning, deep learning, and AI in agriculture.



TAHMINA AKTER received the B.Sc. (Eng.) degree from the University of Chittagong, Chattogram, Bangladesh, and the M.Sc. degree in engineering. She is currently a Senior Lecturer in computer science and engineering with Port City International University, Chattogram. Her research interests include machine learning, deep learning, and AI in agriculture.



MOHAMMAD SHAHADAT HOSSAIN (Senior Member, IEEE) received the M.Phil. and Ph.D. degrees in computation from the Institute of Science and Technology (UMIST), The University of Manchester, U.K., in 1999 and 2002, respectively. He is currently a Professor of computer science and engineering with the University of Chittagong, Bangladesh, and a Visiting Professor with Luleå University of Technology, Sweden. His current research interests include e-government, the modeling of risks and uncertainties using evolutionary computing techniques, the investigation of pragmatic software development tools and methods, information systems in general, and expert systems.



KARL ANDERSSON (Senior Member, IEEE) received the M.Sc. degree in computer science and technology from the Royal Institute of Technology, Stockholm, Sweden, and the Ph.D. degree in mobile systems from Luleå University of Technology, Sweden. After pursuing postdoctoral research with the Internet Real-Time Laboratory, Columbia University, New York, NY, USA, and the National Institute of Information and Communications Technology, Tokyo, Japan, he is currently a Professor with the Cybersecurity Laboratory, Luleå University of Technology. His research interests include green and mobile computing, the Internet of Things, cloud technologies, and information security.